



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

DISEÑO Y DESARROLLO DE SISTEMA DE EVALUACIÓN AUTOMÁTICO DE
EJERCICIOS DE SEGURIDAD OFENSIVA

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

DYLAN NICOLÁS RIQUELME VERGARA

PROFESOR GUÍA:
EDUARDO RIVEROS ROCA

MIEMBROS DE LA COMISIÓN:
ALEJANDRO HEVIA ANGULO
VALENTINA ROJAS OSORIO

SANTIAGO DE CHILE

2026

Resumen

La seguridad en el desarrollo de software es hoy una preocupación central, sin embargo, las malas prácticas y la falta de conocimiento persisten. Aprender de forma práctica cómo funcionan las vulnerabilidades resulta mucho más efectivo que el estudio teórico, pero las plataformas existentes para enseñar seguridad ofensiva suelen depender de *flags* manuales, no ser autohospedables o no ofrecer una validación automática basada en el estado del sistema. Surge así la oportunidad de una plataforma educativa que valide el progreso de forma automática a partir del estado de los sistemas comprometidos.

Este trabajo propone y desarrolla FlexiPwn, una plataforma educativa autohospedable para la práctica de ciberseguridad ofensiva. El educador entrega una imagen vulnerable y define las condiciones de éxito mediante archivos declarativos en YAML. La plataforma instancia un entorno aislado por participante y valida el progreso automáticamente a partir del estado observado, sin *flags* manuales. El sistema se organiza en cuatro capas con responsabilidades delimitadas: los entornos aislados (contenedores Docker en modo *rootless*), el monitoreo pasivo (sistema de archivos, procesos, registros y tráfico de red mediante un *sidecar* con *tcpdump*), un motor de evaluación de reglas declarativas, y la administración a través de una CLI, un REPL interactivo y un *daemon* de seguimiento.

Los objetivos propuestos se cumplieron en su mayoría: la arquitectura modular de cuatro capas, la configuración declarativa, el motor de validación automática basado en estado y las herramientas de seguimiento y administración. El sistema se validó sobre tres escenarios que cubren tres categorías de ejercicio: escalada de privilegios, inyección SQL e inyección de comandos. Además, se incluyeron pruebas automatizadas y verificaciones de contención del entorno, y se puso a disposición de uno de los servidores del CLCERT, aunque sin una validación con un curso real, que queda como principal línea pendiente. Como trabajo futuro se plantea realizar pruebas realistas en un aula real e incorporar más *providers* de aislamiento, escenarios, monitores y *targets*, con la observación a nivel de *kernel* mediante eBPF como una dirección prometedora.

PENDIENTE PENDIENTE PENDIENTE

Agradecimientos

- Cami
- Familia
- Amigos Colegio
- Amigos Universidad
- Eduardo
- CLCERT
- DCC
- Maribel
-

Tabla de Contenido

1. Introducción	1
1.1. Objetivos	2
1.1.1. Objetivo General	2
1.1.2. Objetivos Específicos	2
1.2. Evaluación	2
1.3. Solución Propuesta	3
1.3.1. Arquitectura General	3
1.3.2. Flujo principal	3
1.4. Declaración de uso de inteligencia artificial	4
2. Estado del Arte	6
2.1. Plataformas Comerciales	6
2.2. Plataformas con Validación Automática	6
2.3. Plataformas Estáticas	6
2.4. Plataformas Tipo CTF	7
2.5. Plataformas Avanzadas	7
2.6. Oportunidad Identificada	7
2.6.1. Valor Pedagógico	8
2.6.2. Contribución Técnica	8
3. Diseño	9
3.1. Arquitectura de Capas	9

3.1.1.	Justificación	9
3.1.2.	Descripción de Capas	12
3.2.	Contratos entre Capas	14
3.2.1.	Capa 1 → Capa 2: Estado Observable	14
3.2.2.	Capa 2 → Capa 3: Eventos Estructurados	15
3.2.3.	Capa 3 → Capa 4: Resultados de Evaluación	17
3.2.4.	Capa 4 → Capa 1: Ciclo de Vida	18
3.2.5.	Educador → Capa 3: Archivo YAML del Escenario	18
3.3.	Modelo de Datos	22
3.4.	Aislamiento de Entornos	24
4.	Implementación	26
4.1.	Stack Tecnológico	26
4.1.1.	Lenguaje y gestión de dependencias	26
4.1.2.	Capa 1: Virtualización de Entornos	26
4.1.3.	Capa 2: Monitoreo Pasivo	27
4.1.4.	Capa 3: Motor de Evaluación	27
4.1.5.	Capa 4: Administración	27
4.1.6.	Testing	28
4.1.7.	Resumen del stack	28
4.1.8.	Consideraciones de evolución	28
4.2.	Decisión de Tecnología de Virtualización	29
4.2.1.	Alternativas evaluadas	29
4.2.2.	Comparación ponderada	30
4.2.3.	Decisión: Docker <i>rootless</i> con interfaz abstracta	30
4.2.4.	Trabajo futuro en aislamiento	31
4.3.	Capa 1: Orquestador de Contenedores	31
4.3.1.	Detección del socket Docker	31

4.3.2.	Identificación y etiquetado de recursos	31
4.3.3.	Creación con <i>rollback</i>	32
4.3.4.	Estrategia de <i>baseline</i> para los monitores	33
4.4.	Capa 2: Monitoreo Pasivo	33
4.4.1.	FilesystemMonitor	33
4.4.2.	ProcessMonitor	34
4.4.3.	LogMonitor	34
4.4.4.	NetworkMonitor	34
4.4.5.	Limitaciones del Monitoreo por Polling	36
4.5.	Capa 3: Motor de Evaluación	37
4.6.	Capa 4: Administración	38
4.6.1.	CLI con Typer y Rich	38
4.6.2.	Modelo de <i>daemon</i> : <i>foreground</i> , <i>detach</i> y <i>attach</i>	39
4.6.3.	Coordinación interna: DaemonLoop y SuperMonitor	39
4.6.4.	Vistas del educador: dashboard y feed	41
4.6.5.	Persistencia y ciclo de vida de un <i>run</i>	41
4.6.6.	Credenciales de los participantes	42
4.6.7.	Visor de objetivos y pistas del estudiante	43
4.6.8.	Limpieza del laboratorio	43
4.6.9.	Asignación masiva con batch-start	43
5.	Validación	45
5.1.	Metodología y criterios de validación	45
5.2.	Cobertura del criterio de éxito	46
5.3.	Escenarios validados	46
5.3.1.	Escalada de privilegios vía sudo vim (pwning)	46
5.3.2.	Inyección SQL con <i>bypass</i> de <i>login</i> (web)	47
5.3.3.	Inyección de comandos hacia <i>reverse shell</i> (web)	47

5.4.	Pruebas de robustez del sistema	48
5.4.1.	Casos borde cubiertos por pruebas unitarias	48
5.4.2.	Detección independiente del camino (la “trampa” de tcpdump)	49
5.4.3.	Resistencia y contención del entorno	49
5.5.	Pruebas en infraestructura real	50
5.5.1.	Autoprueba de acceso remoto	50
5.5.2.	Trabajo futuro	50
5.6.	Discusión	50
6.	Conclusión	52
	Bibliografía	55

Índice de Tablas

2.1. Comparación de plataformas de aprendizaje en ciberseguridad.	7
4.1. Componentes del <i>stack</i> tecnológico de FlexiPwn.	28
4.2. Comparación ponderada de tecnologías de virtualización.	30
5.1. Cobertura de categorías y monitores por escenario.	46

Índice de Ilustraciones

3.1.	Arquitectura de cuatro capas de FlexiPwn.	12
3.2.	[Placeholder] Flujo de datos entre las cuatro capas y el educador: estado observable, eventos estructurados, resultados de evaluación, ciclo de vida y YAML del escenario.	22
3.3.	[Placeholder] Diagrama entidad-relación del modelo de datos de FlexiPwn. .	23
3.4.	[Placeholder] Topología de red del entorno: red interna y externa con el <i>sidecar</i> y los tres contenedores. El vulnerable solo en la red interna, el atacante en ambas, y el <i>sidecar</i> compartiendo el <i>netns</i> del vulnerable.	24
4.1.	[Placeholder] Máquina de estados de un <i>run</i> : pending , running , stopping , resetting , completed , failed , timeout y stopped	32
4.2.	[Placeholder] Pipeline de captura de red: tcpdump escribe traffic.txt en un <i>bind mount</i> desde el <i>netns</i> del vulnerable, y el <i>parser</i> emite network_payload y network_connection	35
4.3.	[Placeholder] Componentes del <i>daemon</i> : DaemonLoop y SuperMonitor (con su <i>ThreadPoolExecutor</i>) coordinan <i>N</i> MonitorOrchestrator y los cuatro monitores; el flujo de MonitorEvent pasa por el EvaluationEngine hacia el NotificationSink y la base de datos.	40
4.4.	[Placeholder] Captura del dashboard , el feed y el <i>badge</i> del REPL.	41

Capítulo 1

Introducción

Actualmente, la seguridad en el desarrollo de software es fundamental. Sin embargo, las malas prácticas y la falta de conocimiento siguen existiendo a pesar del aumento de amenazas y la mejora continua de capacidades de los atacantes.

Existe abundante literatura respecto a las vulnerabilidades con las que se puede encontrar un desarrollador, como el OWASP Top 10 [1], que cataloga las vulnerabilidades de software más críticas, incluyendo inyección, pérdida de control de acceso, fallas criptográficas, entre otras. Sin embargo, la mejor forma de conocer el riesgo real de las vulnerabilidades es entendiendo cómo funcionan, para lo cual un enfoque práctico permite validar su correcto entendimiento. Las investigaciones demuestran que el aprendizaje práctico es significativamente más efectivo, con estudios mostrando que los participantes retienen el 75 % de la información a través de práctica *hands-on*, comparado con solo el 5 % mediante métodos teóricos tradicionales [2].

El problema a abordar en este trabajo es facilitar, tanto al equipo docente como a los estudiantes, confirmar sus conocimientos teóricos a partir de una plataforma automatizada que valide sus avances. Las y los desarrolladores necesitan entornos controlados y seguros donde puedan experimentar y explorar vulnerabilidades posibles dentro de una aplicación sin que esta sea de producción. Para ello, se diseñó y desarrolló un sistema de evaluación automático de ejercicios de ciberseguridad ofensiva.

Este proyecto se aproxima al problema desde tres perspectivas complementarias. Desde lo **pedagógico**, se requiere una forma de evaluar el progreso que capture la trayectoria completa del aprendizaje, no solo el resultado final. La validación automática basada en el estado del sistema permite registrar cada paso del estudiante durante el ataque, proporcionando trazabilidad real del proceso de aprendizaje sin recurrir a soluciones artificiales como múltiples *flags* intermedias. Desde lo **técnico**, existe una oportunidad de contribuir al ecosistema de herramientas *open source* de detección de intrusiones, publicando código, patrones y metodologías que permitan que la comunidad académica evalúe y mejore estos enfoques. Desde lo **personal**, este proyecto representa la oportunidad de desarrollar experiencia práctica en el diseño e implementación de sistemas complejos con múltiples componentes, arquitecturas de capas bien definidas, y desafíos de observabilidad cercanos a los que enfrenta la industria de ciberseguridad.

1.1. Objetivos

1.1.1. Objetivo General

Desarrollar una plataforma educativa autohospedable que permita crear entornos de práctica de seguridad ofensiva personalizables, con validación automática del progreso basada en estado de los sistemas comprometidos, facilitando el aprendizaje práctico de vulnerabilidades sin depender de validaciones manuales.

1.1.2. Objetivos Específicos

1. Diseñar e implementar una arquitectura modular que permita desplegar y gestionar múltiples entornos vulnerables simultáneamente mediante tecnologías de virtualización o contenerización, asegurando aislamiento y facilidad de configuración.
2. Desarrollar un sistema de configuración declarativa que permita definir las condiciones de éxito de un ejercicio mediante archivos YAML, haciendo que la plataforma sea accesible incluso a educadores sin conocimiento técnico profundo.
3. Implementar el motor de validación automática que evalúe el estado de los sistemas comprometidos, determinando el progreso sin depender de *flags* predefinidas, ya sea por archivos modificados, servicios comprometidos, usuarios creados, entre otros.
4. Crear herramientas de seguimiento del progreso, acceso a enunciados de ejercicios y administración del sistema para educadores y estudiantes, implementadas inicialmente mediante línea de comandos.
5. Poner la plataforma a disposición del laboratorio de ciberseguridad CLCERT para su evaluación en una instancia real de uso.

1.2. Evaluación

La forma de evaluar este trabajo fue verificando la capacidad del sistema para generar diferentes tipos de ejercicios con validación automática. El trabajo se consideró exitoso si era posible crear ejercicios de seguridad ofensiva con validación automática para al menos tres categorías entre: aplicaciones y servicios web, bases de datos, *pwning*, análisis forense e ingeniería reversa. Se entendió por validación automática aquella que marca el ejercicio como completado automáticamente cuando el participante logra explotar una vulnerabilidad, acceder a un archivo específico, comprometer un servicio u obtener privilegios de usuario, sin requerir la entrada manual de *flags* o códigos predefinidos.

1.3. Solución Propuesta

La solución consiste en una plataforma que permite a los educadores cargar imágenes de sistemas vulnerables y definir reglas de validación mediante archivos YAML, generando automáticamente entornos de práctica personalizados con validación automática.

1.3.1. Arquitectura General

El sistema implementa una arquitectura dividida en cuatro capas con responsabilidades claramente delimitadas. Esta arquitectura emergió del análisis de pruebas de concepto [3] desarrolladas durante el trabajo previo al semestre, donde se identificó que la separación de responsabilidades entre el sistema vulnerable, el mecanismo de detección y la lógica de evaluación es fundamental para lograr flexibilidad y extensibilidad.

Capa 1 — Sistemas aislados Contiene todos los sistemas y redes que componen el escenario de ataque. La implementación actual los materializa como contenedores, aunque la arquitectura es agnóstica al origen de aislamiento y podría sostenerse a futuro sobre tecnologías de virtualización. Se divide en dos subcapas: la Capa 1a, el sistema vulnerable objetivo (el contenedor vulnerable), y la Capa 1b, el entorno atacante desde el cual el estudiante ejecuta el ataque (el contenedor atacante).

Capa 2 — Monitoreo pasivo Observa pasivamente el estado de los contenedores con acceso de lectura externo al sistema. La única excepción a esa lectura externa pura es la captura de tráfico de red, que requiere un contenedor auxiliar (*sidecar*) corriendo *tcpdump*, por ser la forma adecuada de capturar el tráfico del escenario. Los escenarios de referencia asumen tráfico en claro, sin TLS, para que la captura resulte legible. La responsabilidad de esta capa es solo generar un registro estructurado de eventos detectables: modificaciones del sistema de archivos, procesos creados, entradas de log y tráfico o conexiones de red, entre otros. Su comportamiento es análogo al de un SIEM (Security Information and Event Management) acotado a un único escenario de ataque.

Capa 3 — Motor de evaluación Consume los eventos generados por la Capa 2 y evalúa reglas declarativas definidas en archivos YAML que especifican las condiciones de victoria. Determina automáticamente cuándo un participante ha completado objetivos sin requerir la entrada manual de *flags*.

Capa 4 — Administración Gestiona el ciclo de vida completo de los entornos de práctica y mantiene registro del progreso de todos los participantes. Proporciona interfaces de línea de comandos principalmente para educadores, incluyendo vistas agregadas para seguir varios entornos en paralelo.

1.3.2. Flujo principal

1. **Carga de escenarios:** el educador entrega una imagen del sistema vulnerable junto con un archivo YAML que define los objetivos del ejercicio, las condiciones de validación

y los criterios de éxito.

2. **Generación de entornos:** la plataforma instancia la imagen proporcionada, creando un entorno aislado por cada participante que intentará explotar la vulnerabilidad.
3. **Validación automática:** el motor de validación examina constantemente el estado del contenedor, comparando el estado actual contra las reglas definidas en el archivo YAML.
4. **Gestión:** tanto educadores como estudiantes utilizan herramientas de línea de comandos para interactuar con sus respectivos entornos y monitorear el avance.

1.4. Declaración de uso de inteligencia artificial

En la elaboración de esta memoria se utilizaron herramientas de inteligencia artificial generativa, concretamente los modelos Claude de Anthropic (familias Opus y Sonnet 4.x), empleados principalmente a través de la aplicación de escritorio en su modo Cowork. Estas herramientas se usaron a lo largo de todo el trabajo: en la revisión de literatura y la comparación de plataformas, en la discusión de decisiones de arquitectura, en la generación y refactorización de código, en la redacción y edición de este documento, y en tareas de apoyo como el control de versiones y la organización del proyecto. Todo el contenido generado o asistido fue revisado, evaluado y validado por el autor, quien asume plena responsabilidad sobre el trabajo presentado. La inteligencia artificial se empleó como herramienta de apoyo y no se reconoce como coautoría.

El modo Cowork se utilizó como un acelerador del trabajo: permitió delegar la realización del trabajo, como escribir código, redactar y editar, ejecutar y verificar, conservando el rol del autor como arquitecto de las decisiones. Trabajar de esta forma permitió mantener la constancia durante todo el semestre, compatibilizando con las labores del autor como desarrollador de aplicaciones móviles. La integración de la herramienta estuvo acompañado de la verificación: la relectura crítica de cada resultado, la comparación de las afirmaciones y planeamientos contra el código fuente, y el uso de una memoria persistente entre sesiones para mantener el contexto y evitar contradicciones. El criterio principal fue no dar por válido ningún resultado por el solo hecho de que “funciona” a primera vista.

Los prompts siguieron unos patrones recurrentes. Para una tarea de inicio a fin, se partía entregando el contexto (usualmente un resumen de lo trabajado en sesiones anteriores), seguido de lo que había que hacer y de los puntos que valía la pena discutir, pidiendo siempre alternativas para elegir la implementación más adecuada. Se cerraba con algún criterio de validación y con notas que precisaban la ejecución. A mitad de una tarea el patrón cambiaba: se revisaba lo ejecutado y se respondía con un punteo extenso de dudas, cambios deseados y aspectos por considerar, y solo se confirmaba el avance al paso siguiente cuando el resultado era satisfactorio. La mayoría de veces, se recordaba al modelo lo que debería tener en memoria, para asegurar un correcto alineamiento de la tarea a realizar. Los prompts breves se usaban para algunos casos puntuales cuya discusión convenía expandir, y al final de una sesión cargada se solicitaban los cierres habituales: formateo, guardado en memoria, mensaje de *commit* si se requería y un prompt de traspaso que orientara a la sesión siguiente.

A modo ilustrativo, dos ejemplos representativos del tipo de instrucción entregada:

- Un prompt que funcionó bien: “Retoma el informe: primero una foto del estado del repositorio con los commits nuevos. Luego valida afirmación por afirmación contra el código, marcando cada una como verdadera, falsa o imprecisa con su archivo y línea. Entrega un reporte ordenado por criticidad antes de editar y espera mi confirmación. Reglas: 80 caracteres, prosa en presente y opciones ante cada alternativa.” Funcionó porque aportaba contexto, descomponía la tarea, exigía validación y opciones, y fijaba restricciones explícitas.
- Un prompt que funcionó mal: una instrucción amplia y sin contexto como “mejora el capítulo de implementación”, que al no acotar el objetivo ni los criterios de aceptación producía cambios genéricos o de alcance excesivo que luego exigían una corrección considerable. La lección es que los prompts pobres en contexto y sin criterios claros rendían resultados de bajo valor.

Surgió un momento interesante en la etapa final del trabajo, al estudiar una prueba de concepto de escape de contenedores para comprender el funcionamiento de este tipo de exploit y respaldar el análisis de contención del entorno. Al tratar un tema sensible, las primeras consultas sobre una prueba de concepto realista del CVE [4, 5] en cuestión fueron rechazadas por las políticas de uso de la herramienta. Tras solicitar al equipo de Anthropic un acceso ampliado y explicar su propósito educativo dentro de este trabajo de ciberseguridad, la consulta pudo retomarse. A partir de ahí, el modelo resultó de gran ayuda para seguir el exploit paso a paso, consiguiendo compararlo con otros CVE relacionados, distinguiendo, por ejemplo, los que se ejecutan desde el interior del contenedor de los que afectan directamente al *host*.

Si el trabajo se reiniciara, se cambiarían algunas formas del uso de la inteligencia artificial. Se establecería antes una rutina de revisión y verificación sistemática, para no depender de que un resultado “parezca” correcto. Además, se sería más disciplinado con la memoria persistente y los traspasos entre sesiones, que resultaron clave para mantener la coherencia a lo largo de un trabajo asistido por IA.

Capítulo 2

Estado del Arte

En la actualidad existe una amplia variedad de plataformas que apoyan el aprendizaje en ciberseguridad. La mayoría recurre a la ludificación con el objetivo de facilitar el entendimiento mediante ejercicios prácticos, tanto de seguridad ofensiva como defensiva.

2.1. Plataformas Comerciales

Entre las plataformas comerciales más reconocidas están TryHackMe (THM) [6] y Hack The Box (HTB) [7], las cuales poseen contenido de alta calidad y ofrecen cursos y certificaciones en ciberseguridad. Ambas comparten el método de conexión a entornos hospedados por ellos y validación de *flags* para sus módulos de aprendizaje, lo que eventualmente facilita la trampa si no se toman precauciones adecuadas.

2.2. Plataformas con Validación Automática

PortSwigger Web Security Academy [8] es una plataforma que se especializa en enseñar seguridad de aplicaciones web. Es de las pocas que incluye un sistema automático para detectar avances al explotar una vulnerabilidad, sin requerir validación manual mediante *flags*. No obstante, no es autohospedable y está enfocada exclusivamente a vulnerabilidades web no personalizables.

2.3. Plataformas Estáticas

Damn Vulnerable Web Application (DVWA) [9] es un buen método para aprender seguridad web, pero no tiene capacidad de generar escenarios personalizados de forma directa ni seguimiento de progreso, y se valida con mensajes simples al explotar cierta vulnerabilidad.

2.4. Plataformas Tipo CTF

CTFd [10], RootTheBox [11] y picoCTF [12] son herramientas para preparar competencias de captura la bandera (*capture the flag*), con roles pedagógicos como el del curso CC5325 Taller de Hacking Competitivo. Sin embargo, para validar un ejercicio resuelto también requieren ingresar *flags* manualmente, permitiendo posibles copias entre estudiantes.

2.5. Plataformas Avanzadas

Existen opciones más complejas como CyberRangeCZ [13], EDURange [14] o Tectonic [15] para simular campos de entrenamiento de ciberseguridad (*cyber ranges*), los cuales requieren mayor conocimiento de infraestructura técnica, lo que los hace opciones poco accesibles para contextos educativos pequeños.

2.6. Oportunidad Identificada

El análisis comparativo presentado en la Tabla 2.1 demuestra que no existe una solución que integre validación automática basada en estado del sistema, sea autohospedable y sea completamente personalizable. Esta combinación es un aporte relevante para quien desee generar instancias de aprendizaje de seguridad ofensiva de forma rápida, donde baste cambiar la imagen del sistema para modificar completamente el objetivo de la actividad.

Para facilitar la comparación, se establecieron cuatro criterios clave: **Autohospedable** se refiere a la capacidad de instalar y ejecutar la plataforma en infraestructura propia; **Gratuito** indica si se puede usar sin costo (*Freemium* implica funcionalidades limitadas gratuitas); **Validación** describe el método para verificar ejercicios completados; y **Personalizable** evalúa la capacidad de crear contenido propio.

Tabla 2.1: Comparación de plataformas de aprendizaje en ciberseguridad.

Plataforma	Autohospedable	Gratuito	Validación	Personalizable
TryHackMe	No	Freemium	Flags	No
Hack The Box	No	Freemium	Flags	No
PortSwigger	No	Gratuito	Automática	No
DVWA	Sí	Gratuito	Manual	Limitado
CTFd	Sí	Gratuito	Flags	Sí
RootTheBox	Sí	Gratuito	Flags	Sí
picoCTF	Sí	Gratuito	Flags	Sí
CyberRangeCZ	Sí	Gratuito	Multicriterio	Sí
EDURange	Sí	Gratuito	Multicriterio	Sí
Tectonic	Sí	Gratuito	Multicriterio	Sí
Propuesta	Sí	Gratuito	Automática	Sí

2.6.1. Valor Pedagógico

La validación automática basada en estado ofrece ventajas pedagógicas significativas sobre los sistemas tradicionales de *flags*. Mientras que las *flags* solo indican si un estudiante alcanzó un objetivo final, un sistema de monitoreo de estado registra la evolución completa del ataque: qué payloads envió por la red, qué vulnerabilidades intentó explotar, o qué archivos modificó. Esta trazabilidad permite a los educadores entender no solo si un estudiante tuvo éxito, sino **cómo** lo logró, detectar dificultades comunes y proporcionar retroalimentación más específica y formativa.

2.6.2. Contribución Técnica

FlexiPwn no pretende ser una contribución mayor al ecosistema de sistemas de detección y prevención de intrusiones (IDS/IPS), pero sí aporta al ámbito *open source* de herramientas educativas de detección. Los sistemas comerciales de detección suelen mantener sus reglas como propiedad intelectual cerrada, lo que dificulta que la comunidad las evalúe y mejore. Un proyecto educativo que implementa detección basada en reglas declarativas, con código completamente abierto y auditable, permite que investigadores y profesionales estudien, adapten y evolucionen estos enfoques.

El enfoque de reglas declarativas de FlexiPwn se inspira en formatos abiertos de firmas como SIGMA [16], pensado para describir detecciones de forma portable sobre soluciones de gestión de eventos de seguridad (SIEM), que en el ámbito comercial suelen depender de lenguajes de reglas propietarios. FlexiPwn adopta esa idea de firmas legibles y abiertas, pero acotada a un alcance educativo y al monitoreo de estado de los escenarios.

Capítulo 3

Diseño

Este capítulo describe las decisiones de diseño del sistema FlexiPwn, incluyendo la arquitectura de capas, los contratos entre ellas, el modelo de datos y el modelo de aislamiento de entornos.

3.1. Arquitectura de Capas

3.1.1. Justificación

La decisión de estructurar el sistema en capas responde a principios arquitectónicos que se identificaron como necesarios a partir de las pruebas de concepto desarrolladas previo al semestre. Las limitaciones encontradas convergieron hacia un patrón común: la mezcla de responsabilidades entre el sistema a vulnerar, el mecanismo de detección y la lógica de evaluación resultaba en implementaciones frágiles, poco extensibles y fuertemente acopladas a tecnologías específicas. A partir de este análisis emergieron cuatro principios que fundamentan la arquitectura de cuatro capas.

Separación de responsabilidades

Cada capa tiene una función específica y bien definida, lo que facilita el desarrollo, *testing* y mantenimiento independiente de cada componente. Esta separación permite trabajar en capas distintas sin interferencia, y simplifica la depuración cuando surgen problemas. En las pruebas de concepto previas, la generación de eventos y la evaluación de condiciones de éxito convivían en el mismo módulo, lo que hacía que agregar un nuevo tipo de fuente de eventos o cambiar la lógica de evaluación requiriera modificar el mismo archivo, aumentando el acoplamiento y la fragilidad del sistema. En FlexiPwn, estas responsabilidades están explícitamente separadas: la Capa 2 solo observa y genera eventos, la Capa 3 solo evalúa condiciones, y ninguna de las dos modifica el estado de los entornos.

Reutilización y flexibilidad

Al abstraer las capas inferiores mediante interfaces bien definidas, el sistema puede soportar múltiples tecnologías de virtualización sin modificar las capas superiores de validación y administración. Esto permite adaptar el sistema a diferentes contextos e infraestructuras. La interfaz abstracta `EnvironmentProvider` encapsula toda la complejidad necesaria para distintas tecnologías de virtualización, como Docker *rootless*: cambiar la tecnología de virtualización solo requiere implementar esa interfaz en una nueva clase, sin tocar el motor de evaluación ni la CLI de administración. Por ejemplo, una futura implementación `PodmanProvider` o `KVMPProvider` no tendría impacto en las capas 3 y 4.

Extensibilidad

Nuevos tipos de validaciones o métodos de monitoreo pueden agregarse a las capas 2 y 3 sin modificar los sistemas vulnerables ni la administración. Por ejemplo, agregar soporte para detectar exfiltración de datos mediante DNS *tunneling* solo requiere modificar la Capa 2 para capturar ese tipo de eventos y la Capa 3 para evaluar las reglas correspondientes. El diseño de registro de evaluadores mediante un patrón de *registry* en la Capa 3 permite añadir nuevos tipos de *target* sin modificar el motor de evaluación central.

Esta extensibilidad opera principalmente sobre las capas 2 y 3, pero algunos métodos de extracción de información requieren ampliar también la interfaz expuesta por la Capa 1. La incorporación del monitor de red, por ejemplo, exigió agregar el despliegue de un contenedor *sidecar* de captura al `EnvironmentProvider`. Un futuro monitor basado en eBPF probablemente requiera exponer un nuevo método para adjuntar programas al *cgroup* del contenedor vulnerable. La interfaz abstracta admite estos crecimientos siempre que se respete el principio de pasividad: la Capa 1 expone capacidades de observación, no las consume ni las interpreta.

Agnóstico a la implementación

Los educadores que crean desafíos no necesitan conocer los detalles de cómo funciona la detección, solo deben proporcionar una imagen vulnerable y un archivo YAML con las reglas declarativas. Esto reduce significativamente la barrera de entrada para crear contenido educativo. De igual forma, los sistemas vulnerables de Capa 1 no saben que están siendo monitoreados, lo cual es un principio crítico para la validez pedagógica del ejercicio: el entorno debe comportarse exactamente como lo haría en producción, sin cooperar con el sistema de detección. Este principio fue el aprendizaje más importante extraído de las pruebas de concepto previas, donde la lógica de detección estaba parcialmente implementada dentro del sistema vulnerable, violando la separación y limitando la aplicabilidad del enfoque.

No obstante, ser agnóstico a la implementación no implica ser independiente del tipo de aplicación. La eficacia de cada método de detección sigue dependiendo de las características del sistema vulnerable expuesto. La detección de inyección SQL (`log_pattern` + `network_payload`), por ejemplo, asume que el motor de base de datos elegido permite

exponer un *query log* legible por el monitor (caso de MySQL con `general_log`) y, cuando se quiere inspeccionar el payload SQL a nivel de red, que la conexión *cliente-servidor* opere sin TLS en el escenario, dado que el monitor de red solo expone metadatos sobre tráfico cifrado. Un escenario que use una base de datos que no exponga logs equivalentes obligaría al educador a definir condiciones de éxito por otras señales. Esta dependencia es propia del dominio del ejercicio y no del sistema de detección: el motor sigue ejecutando reglas declarativas sin conocer detalles de la aplicación, pero la elección de qué reglas son verificables sí depende de qué señales emite el sistema vulnerable.

***Trade-offs* de la arquitectura**

Esta arquitectura introduce complejidad adicional en comparación con un diseño monolítico, particularmente en la coordinación entre capas, la necesidad de definir interfaces claras y el *overhead* de comunicación entre componentes. Sin embargo, los beneficios en términos de mantenibilidad, portabilidad y capacidad de evolución del sistema justifican esta complejidad inicial, especialmente considerando que se espera que el sistema crezca con nuevos tipos de validadores, tecnologías de virtualización y escenarios de ejercicios a futuro.

3.1.2. Descripción de Capas

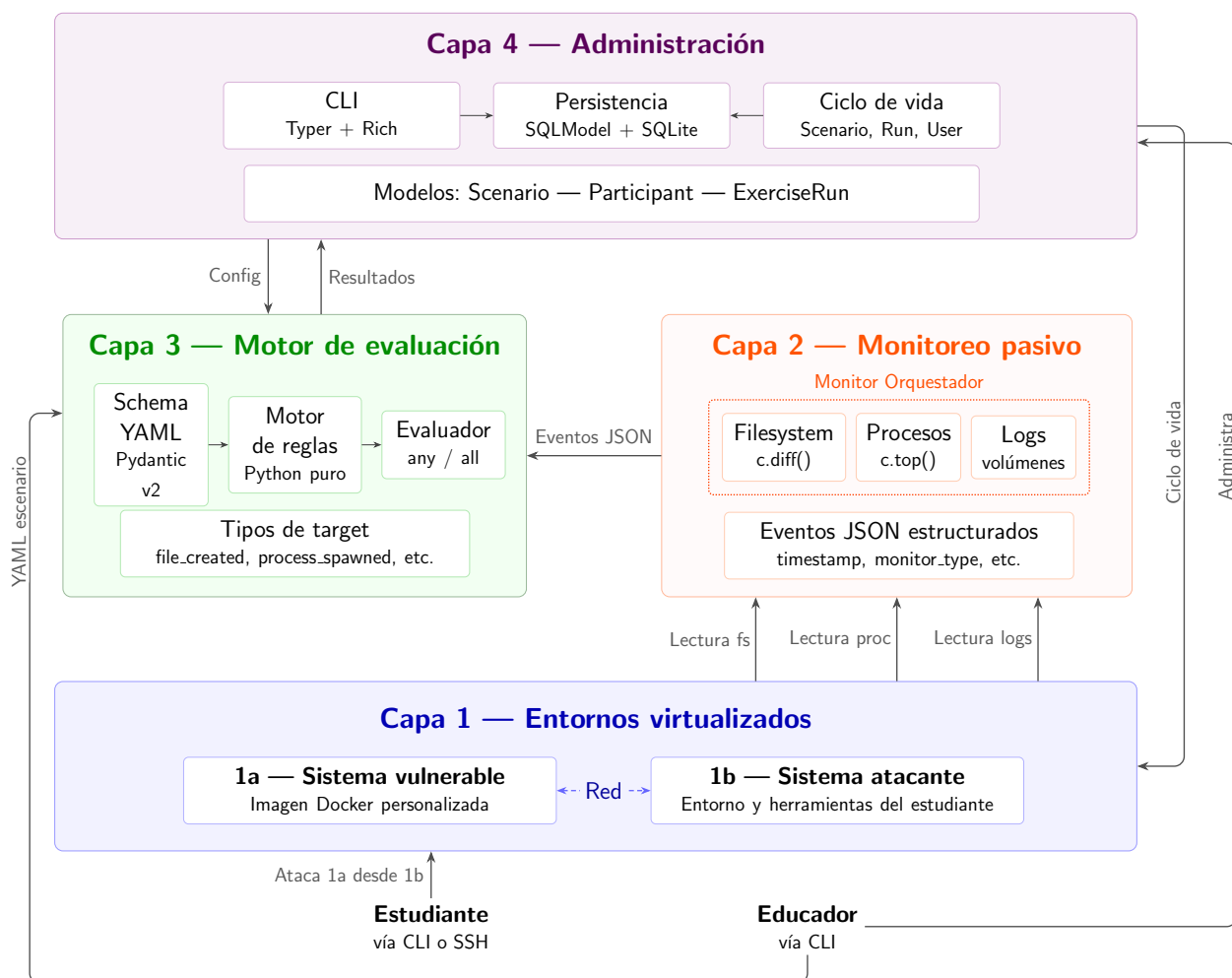


Figura 3.1: Arquitectura de cuatro capas de FlexiPwn.

Capa 1: Sistemas Virtualizados

Contiene en estado virtualizado todos los sistemas y redes que componen el escenario de ataque. Se divide en dos subcapas con roles diferenciados pero complementarios.

La **Capa 1a** representa el sistema vulnerable objetivo. Esta subcapa autocontiene todo el estado (archivos, memoria, procesos) necesario para determinar si un ataque fue exitoso. Es accesible al participante pero no comparte información de forma activa con otras capas del sistema, y no sabe que está siendo monitoreada.

La **Capa 1b** proporciona al participante un entorno con herramientas para ejecutar ataques. No tiene más acceso al sistema vulnerable que el necesario para ejecutar el ataque, típicamente conexión a la misma red interna.

Capa 2: Generación de Eventos

Tiene acceso de lectura externo al estado de los contenedores de Capa 1. Su única responsabilidad es generar un registro estructurado de eventos de seguridad detectables: cambios en el sistema de archivos, nuevos procesos, entradas de log de aplicación y tráfico de red observado. Su comportamiento es cercano al de un SIEM acotado a un único escenario de ataque.

El principio fundamental de esta capa es la **pasividad total**: no ejecuta ningún comando dentro del contenedor vulnerable ni modifica su estado. Los mecanismos de observación son completamente externos:

- **Filesystem**: mediante `container.diff()` del Docker SDK, que retorna todos los archivos modificados, creados o eliminados respecto a la imagen original, sin montar volúmenes en el contenedor.
- **Procesos**: mediante `container.top()` con argumentos `-o pid,euid,ppid,cmd`, que lee la información de procesos desde el kernel del host sin ejecutar nada dentro del contenedor.
- **Logs de aplicación**: mediante lectura directa de archivos de log en el host, escritos por el contenedor a través de volúmenes montados en rutas explícitamente definidas en el YAML del escenario.
- **Red**: mediante un contenedor *sidecar* de captura (basado en `nicolaka/netshoot`) que comparte el *network namespace* del contenedor vulnerable. El *sidecar* ejecuta `tcpdump` y vuelca el resultado a un archivo en un *bind mount* del host. El monitor de red lee ese archivo desde fuera del contenedor vulnerable y emite los eventos correspondientes. El contenedor vulnerable nunca interactúa con `tcpdump` ni sabe que su tráfico se está observando.

Los cuatro monitores son coordinados por un *orquestador* que provee un único ciclo de *polling* por entorno, agrupa los eventos emitidos y los entrega en orden cronológico a la Capa 3. El detalle de esta coordinación, incluyendo el orquestador global que supervisa varios entornos en paralelo, corresponde al capítulo 4 de implementación.

Capa 3: Motor de Evaluación

Consume los eventos generados por la Capa 2 y evalúa condiciones de victoria definidas mediante reglas declarativas en archivos YAML. Determina cuándo un participante ha completado exitosamente un ejercicio sin requerir la entrada manual de *flags*. Esta capa no tiene acceso directo a los contenedores: todo su conocimiento del mundo exterior llega exclusivamente por los eventos de Capa 2.

Capa 4: Administración

Gestiona el ciclo de vida de los entornos: crear, reiniciar o destruir según sea necesario. Mantiene registro del progreso de todos los participantes según lo indicado por la Capa 3 y proporciona interfaces de línea de comandos para educadores y estudiantes.

3.2. Contratos entre Capas

Los contratos definen qué datos cruzan cada frontera, en qué formato, quién los produce y quién los consume. El objetivo es que cada capa pueda desarrollarse, testearse y reemplazarse de forma independiente siempre que respete estos contratos.

3.2.1. Capa 1 → Capa 2: Estado Observable

La Capa 1 no comunica nada activamente a la Capa 2. Este es un contrato implícito: la Capa 2 lee el estado de la Capa 1, y la Capa 1 no sabe que está siendo observada. La interfaz entre ambas es el estado observable del entorno virtualizado, accedido a través de tres canales descritos en la Sección anterior.

Toda interacción con los contenedores se canaliza a través de la interfaz abstracta `EnvironmentProvider`, que encapsula la tecnología de virtualización:

```
class EnvironmentProvider(ABC):
    def create(self, scenario_id, participant_id, image,
               attacker_image=None, ports=None, attacker_ports=None,
               timeout_seconds=1800, startup_delay=None,
               enable_network_capture=False,
               capture_filter="") -> Environment
    def destroy(self, env_id) -> None
    def reset(self, env_id) -> None
    def get_status(self, env_id) -> Environment
    def exec_run(self, env_id, cmd, ...) -> ExecResult
    def get_filesystem_diff(self, env_id) -> list[dict]
    def get_processes(self, env_id) -> list[ProcessInfo]
    def cleanup_all(self) -> None
```

En el estado actual del sistema la única implementación concreta de esta interfaz es `DockerRootlessProvider`, que utiliza el SDK oficial de Docker (`docker-py`) sobre un *daemon* en modo *rootless*. Este proveedor materializa cada `Environment` como un conjunto de recursos Docker etiquetados con `flexipwn.managed=true` y `flexipwn.env_id`. Mantiene los nombres de contenedores y redes derivados del `env_id` para permitir reconciliación tras una caída del proceso. Una implementación alternativa (`PodmanProvider`, `KVMPProvider`, etc.)

solo debe respetar la firma de la interfaz para que las capas superiores puedan operarla sin cambios.

3.2.2. Capa 2 → Capa 3: Eventos Estructurados

La Capa 2 produce eventos JSON estructurados que la Capa 3 consume. Este es el contrato más crítico del sistema: define el lenguaje común entre la observación y la evaluación. Todos los eventos comparten un envelope común validado por Pydantic v2:

```
class MonitorEvent(BaseModel):
    timestamp: datetime
    monitor_type: Literal["filesystem", "process", "log", "network"]
    event_type: str
    env_id: str
    participant_id: str
    scenario_id: str
    details: dict[str, Any]
```

El campo `monitor_type` identifica la fuente del evento y `event_type` especifica la naturaleza del cambio. El campo `details` varía según el monitor que produce el evento.

Eventos de filesystem

Generados por el monitor de filesystem a partir de `container.diff()`. Los `event_type` posibles y los campos de `details` asociados son:

- **file_created**: archivo nuevo detectado respecto a la imagen original. `details` contiene `path` (ruta absoluta) y `kind` (valor 1, según la convención de `container.diff()`).
- **file_modified**: archivo existente en la imagen que fue modificado. `details` contiene `path` y `kind` (valor 0).
- **file_deleted**: archivo de la imagen original eliminado durante el ejercicio. `details` contiene `path` y `kind` (valor 2). Cabe notar que este tipo de evento no tiene un target correspondiente en el schema YAML: ningún escenario del sistema evalúa la eliminación de archivos como condición de éxito en el estado actual.

Eventos de procesos

Generados por el monitor de procesos a partir de `container.top()`. El único `event_type` implementado es:

- **process_spawned**: proceso nuevo detectado respecto al *baseline* capturado al iniciar el monitor. Los campos de **details** son **pid**, **euid** (effective UID del proceso), **ppid**, **cmd** (comando completo), **lstart** (timestamp de inicio del proceso en formato "Mon Oct 23 10:25:44 2023"), **ppid_cmd** (comando del proceso padre directo), **ancestor_cmds** (lista de comandos de los ancestros, del más cercano al más lejano) y **process_id** (hash SHA-256 truncado a 12 caracteres de la combinación **pid:lstart**, que identifica claramente al proceso incluso ante reutilización de PID gracias al timestamp de inicio, pero cuando **lstart** no está disponible se recurre a **pid:cmd**). Los campos **ppid_cmd** y **ancestor_cmds** son los que alimentan los filtros **ppid_cmd_contains** y **ancestor_contains** de la Capa 3.

Eventos de logs

Generados por el monitor de logs mediante lectura directa de archivos en el host escritos a través de volúmenes Docker.

- **log_entry**: línea de log nueva detectada en un archivo monitoreado. El monitor auto-detecta el formato de cada línea, lo que produce dos variantes de **details**. Si la línea parsea como JSON, contiene **source_file** (ruta del archivo de log en el host) y **parsed** (objeto JSON completo de la línea). Si no, la línea se trata como texto plano y **details** contiene **source_file** y **raw_line** (la línea íntegra). Esta segunda variante es la que consumen los escenarios basados en MySQL, cuyo *query log* es texto plano.

Eventos de red

Generados por el **NetworkMonitor** a partir de la captura de tráfico realizada por el contenedor *sidecar* basado en **nicolaka/netshoot** (descrito en la Sección 4.4.4). Se emiten dos tipos de eventos:

- **network_payload**: paquete observado con contenido ASCII imprimible relevante. **details** contiene **data** (texto ASCII imprimible extraído del payload, la captura se realiza a nivel de paquete con **tcpdump -A**, y el parser observa además las banderas TCP del paquete, por ejemplo **[S.]** o **[P.]**, para clasificarlo), **src_ip**, **src_port**, **dst_ip** y **dst_port**.
- **network_connection**: conexión TCP establecida hacia un destino, detectada por el primer paquete **SYN-ACK** (Flags **[S.]**). **details** contiene **src_ip**, **dst_ip** y **dst_port**.

La Capa 2 garantiza que todo evento emitido cumple el schema antes de enviarse, que los eventos se emiten en orden cronológico por monitor, y que no se emiten duplicados para el mismo cambio de estado.

3.2.3. Capa 3 → Capa 4: Resultados de Evaluación

La Capa 3 comunica a la Capa 4 los resultados de la evaluación de condiciones de éxito mediante dos estructuras Pydantic complementarias.

`TargetResult` representa el estado de evaluación de un objetivo individual:

```
class TargetResult(BaseModel):
    target_index: int
    target_type: str
    description: str
    matched: bool = False
    matched_at: datetime | None = None
    trigger_event: MonitorEvent | None = None
    children: list["TargetResult"] | None = None
```

El campo `trigger_event` contiene el `MonitorEvent` exacto que causó el *match*, lo que permite al educador reconstruir no solo qué objetivo se completó, sino qué acción concreta del estudiante lo disparó. Esta trazabilidad es uno de los valores pedagógicos diferenciales del sistema respecto a los sistemas de *flags*.

El campo `children` se utiliza únicamente cuando el target es un nodo lógico (`and`, `or` o `not`). Contiene los `TargetResult` de los sub-targets evaluados recursivamente, lo que permite a la Capa 4 mostrar al educador no solo qué rama lógica *matcheó*, sino qué sub-condiciones internas fueron las que la satisficieron.

`EvaluationResult` agrega el estado de todos los targets de un escenario para un participante:

```
class EvaluationResult(BaseModel):
    scenario_id: str
    participant_id: str
    env_id: str
    condition: str # "any" | "all"
    targets: list[TargetResult]
    completed: bool
    completed_at: datetime | None = None
    progress: float # hojas matcheadas / total hojas
```

El campo `progress` se calcula como la razón entre el número de *hojas* matcheadas y el total de hojas del árbol de targets del escenario. Las hojas son los targets atómicos (`file_created`, `process_running`, `network_payload`, etc.). Los nodos lógicos (`and`, `or`, `not`) no cuentan como hoja: aportan estructura pero no se incluyen en el denominador del progreso. El campo `completed` es `True` si y solo si la condición (`any` o `all`) se satisface con el conjunto de targets de primer nivel marcados como `matched`. Cuando un target de primer

nivel es un nodo lógico, su `matched` se determina recursivamente: `and` matchea si todos sus hijos matchean, `or` si al menos uno, `not` si su único hijo no matchea.

El `EvaluationEngine` invoca el callback `on_update` cada vez que un target cambia de estado o cuando `completed` pasa de `False` a `True`. Los logros son irreversibles dentro de un *run*: una vez que `matched` es `True`, no vuelve a `False`. La Capa 4 consume estos resultados para persistirlos en SQLite mediante la entidad `TargetResult` y para emitirlos a través de un `NotificationSink` central que decide, según política por tipo, qué se imprime en el TTY del educador y qué queda únicamente en log y en las vistas agregadas `dashboard` y `feed`. A partir de los instantes en que cada hoja matchea, la Capa 4 deriva un registro de tiempos por run: la duración total y el tiempo de cada etapa hasta su objetivo, que se ofrece al educador en esas mismas vistas sin necesidad de almacenar datos adicionales.

3.2.4. Capa 4 → Capa 1: Ciclo de Vida

La Capa 4 es la única capa que modifica el estado de la Capa 1, y lo hace exclusivamente a través de los métodos de la interfaz `EnvironmentProvider`: la materialización de un entorno, su reinicio, su detención o destrucción, y la reconciliación del estado persistido con los recursos reales del proveedor. Estas operaciones son invocaciones internas que la Capa 4 encadena para sostener los flujos expuestos al usuario. El detalle de cómo cada acción del educador o del estudiante se traduce en una secuencia de invocaciones al `EnvironmentProvider` se describe en el capítulo de implementación.

Las consultas de estado (`get_status`, `get_processes`, `get_filesystem_diff`) son de solo lectura y, aunque la Capa 4 también las utiliza para producir vistas al usuario, conviven con el uso que de ellas hace la Capa 2 para generar eventos. La separación se preserva porque ambas capas consumen información sin modificar el entorno.

3.2.5. Educador → Capa 3: Archivo YAML del Escenario

El educador define un escenario mediante un único archivo YAML que la Capa 3 parsea y valida con Pydantic v2 antes de cualquier ejecución. Este archivo es la única pieza de configuración que el educador necesita escribir, además de la imagen Docker del sistema vulnerable.

La estructura completa del archivo es la siguiente:

```
title: str                # nombre del ejercicio (requerido)
description: str          # descripción del ejercicio (requerido)
author: str               # autor del escenario (requerido)
level: str                # beginner | intermediate | advanced
category: str             # pwning | web | database | forensics | reversing

environment:
  image: str               # imagen Docker para Capa 1a (requerido)
```

```

attacker_image: str      # imagen Docker para Capa 1b (requerido)
log_paths: list[str]     # rutas de logs a monitorear (opcional)
ports: list[str]         # puertos extra del vulnerable
                        # "host:container" (opcional)
capture_filter: str      # filtro para tcpdump, ej. "port 3306"
                        # (opcional, solo aplica si hay targets
                        # de tipo network_*)
startup_delay_seconds: float # delay opcional antes de capturar
                        # el baseline si no hay HEALTHCHECK
                        # (null usa el default global)

hints:
  - str                  # pistas ordenadas para el estudiante (opcional)

targets:                 # lista de objetivos (requerido, mínimo 1)
  - type: str
    description: str
    # campos específicos según el tipo de target

condition: str           # any | all (requerido, aplica solo a la
                        # lista plana de targets, se ignora cuando
                        # los targets de primer nivel son nodos
                        # lógicos and/or/not)
timeout_seconds: int     # tiempo máximo en segundos (default: 1800)

```

Los tipos de target disponibles se dividen en *tipos atómicos*, que evalúan una condición concreta sobre eventos de la Capa 2, y *nodos lógicos*, que componen otros targets en árboles recursivos.

Tipos atómicos

file_created Detecta la creación de un archivo nuevo. Requiere **path** (ruta absoluta o directorio terminado en /). Acepta **pattern** (glob) cuando **path** es un directorio.

file_modified Detecta la modificación de un archivo existente en la imagen original. Requiere **path** (ruta absoluta).

process_running Detecta un proceso activo con características específicas. Requiere **euid** (effective UID esperado, 0 para root) y **cmd_contains** (substring en el comando del proceso). Acepta **ppid_cmd_contains** (substring en el comando del proceso padre) y **ancestor_contains** (substring en el comando de cualquier ancestro).

log_pattern Detecta una entrada de log que cumpla condiciones sobre campos parseados. Requiere **field_matches** (diccionario de campo → regex). El campo especial **raw_line** permite matchear contra la línea completa cuando el log es texto plano. Requiere que **environment.log_paths** declare al menos una ruta.

network_payload Detecta payloads de paquetes de red cuyo contenido ASCII imprimible coincida con una expresión regular. Requiere **field_matches** con la clave **data**, y el matching se realiza con **re.IGNORECASE** sobre el texto extraído del paquete. El *sidecar* de captura comparte el *network namespace* del contenedor vulnerable. Opcionalmente se acota la captura con **capture_filter** en el bloque **environment**.

network_connection Detecta conexiones TCP establecidas (paquete SYN-ACK) hacia un destino. Requiere **dst_port**, y acepta **dst_ip** para restringir a una IP específica.

Nodos lógicos

Los nodos lógicos componen otros targets en árboles recursivos y se identifican por su **type**. Sus sub-targets se definen en el campo **targets**, que puede contener tanto tipos atómicos como otros nodos lógicos.

and Matchea cuando todos sus sub-targets matchean. Requiere al menos dos sub-targets.

or Matchea cuando al menos un sub-target matchea. Requiere al menos dos sub-targets.

not Matchea cuando su único sub-target no matchea. Requiere exactamente un sub-target. No puede aparecer como target de primer nivel del escenario, ya que un escenario que se completara por la ausencia de un evento podría darse por exitoso sin acción alguna del estudiante. Este caso se valida al cargar el YAML.

El campo de primer nivel **condition** (**any** o **all**) actúa como azúcar sintáctica para listas planas de targets atómicos: es equivalente a envolver la lista en un nodo **or** o **and** respectivamente.

A modo de ejemplo, el siguiente archivo YAML define un escenario de escalación de privilegios mediante **sudo vim**:

```
title: "Privilege escalation via sudo vim"
description: >
  El usuario ctfuser tiene permisos sudo sobre vim sin contraseña.
  Escala privilegios y crea evidencia de acceso root.
author: "Dylan Riquelme"
level: beginner
category: pwning

environment:
  image: "flexipwn/vuln-sudo"
  attacker_image: "flexipwn/attacker"
  log_paths: []
  ports: []
  startup_delay_seconds: 3
```

```

hints:
- "Conéctate al vulnerable desde el atacante: ssh ctfuser@flexipwn-<env_id>-vulnerable"
- "Una vez dentro, revisa tus permisos sudo con: sudo -l"
- "Vim puede ejecutar comandos del sistema con :!comando"

targets:
- type: file_created
  path: "/root/"
  pattern: "*.txt"
  description: "Archivo .txt creado en directorio root"

- type: file_modified
  path: "/etc/passwd"
  description: "Archivo de usuarios del sistema modificado"

- type: process_running
  euid: 0
  cmd_contains: "bash"
  ppid_cmd_contains: "vim"
  ancestor_contains: "sudo"
  description: "Shell root lanzada desde vim (escalación exitosa)"

condition: all
timeout_seconds: 1800

```

El schema se valida al momento de cargar el escenario, antes de cualquier ejecución. Los errores de validación producen mensajes que indican el campo problemático y el valor esperado, sin requerir ejecutar el sistema para descubrirlos.

Más allá de la carga de escenarios, este contrato también da soporte a las operaciones de validación y limpieza que el educador necesita sobre los escenarios y la actividad acumulada. Se exponen más detalles en el capítulo de implementación.

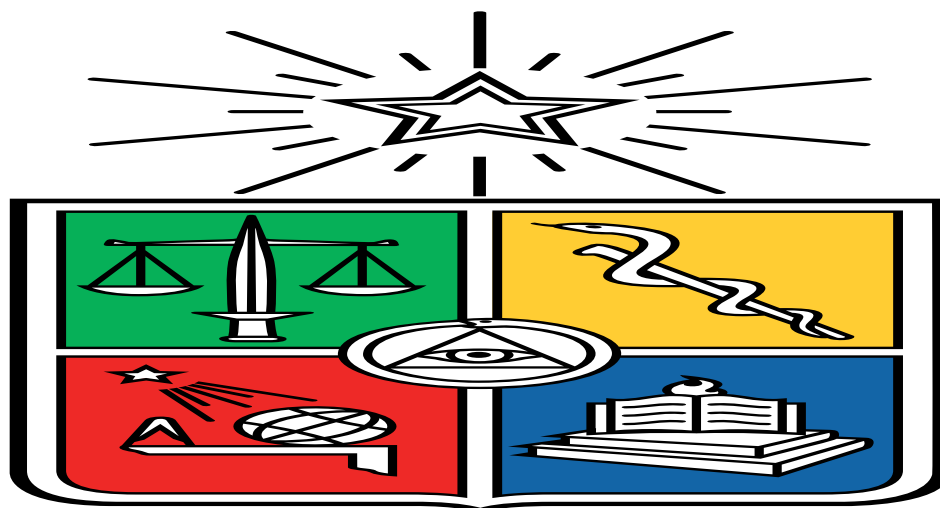


Figura 3.2: [Placeholder] Flujo de datos entre las cuatro capas y el educador: estado observable, eventos estructurados, resultados de evaluación, ciclo de vida y YAML del escenario.

3.3. Modelo de Datos

El estado del sistema se persiste en una base de datos SQLite gestionada mediante SQL-Model. La elección de SQLite responde al contexto de despliegue: una instalación autohospedada de escala pequeña a mediana que no justifica la operación de un servidor de base de datos independiente.

El modelo contempla cinco entidades principales. Todas usan identificadores UUID como clave primaria, generados automáticamente al persistir, lo que evita filtrar información temporal en los identificadores y simplifica la fusión de bases de datos entre instalaciones distintas.

Scenario almacena los metadatos del escenario cargado por el educador, incluyendo la ruta al archivo YAML, el contenido íntegro del YAML para reproducibilidad, y campos desnormalizados (`title`, `description`, `level`, `category`, `image`, `attacker_image`, `timeout_seconds`) para consultas directas sin reparsear el YAML.

Participant representa a un estudiante del sistema. Tiene `username` con índice único, evitando colisiones a nivel de base de datos. La contraseña se persiste como `hash` bcrypt, el texto plano solo se expone una vez al momento de crear o resetear el participante, y nunca se vuelve a recuperar.

ExerciseRun vincula a un participante con un escenario y registra el estado del ejercicio. Mantiene índices ordinarios sobre las claves foráneas `scenario_id` y `participant_id` para acelerar las consultas frecuentes (runs por participante, runs por escenario), y un índice único sobre `env_id` para garantizar que el identificador de entorno expuesto al participante no se repita entre runs. Persiste además los datos efímeros de acceso del estudiante (`attacker_ssh_port`, `attacker_ssh_username`, `attacker_ssh_password`) para que el educador pueda recuperar las credenciales en cualquier momento desde la CLI. La contraseña SSH

se guarda en texto plano dentro del archivo SQLite del educador, con la trazabilidad documentada en el código sobre por qué este compromiso es aceptable para el contexto educativo. El campo `status` recorre los estados de `RunStatus` a lo largo del ciclo de vida del ejercicio (desde `pending` hasta los estados terminales `completed`, `failed`, `timeout` o `stopped`, pasando por `running`, `stopping` y `resetting`). Mientras que los campos `reset_payload` y `daemon_message` acompañan a esas transiciones mediadas por el *daemon*. La validación de unicidad sobre el par (`scenario_id`, `participant_id`) para runs activos se hace a nivel aplicación, no de base de datos: el mismo participante puede tener múltiples runs históricos del mismo escenario, lo que permite reintentos sin borrar el historial previo.

TargetResult registra el estado de evaluación de cada objetivo individual dentro de un *run*, incluyendo el evento exacto que causó el *match* serializado como JSON. Cada fila corresponde a una *hoja* del árbol de targets: el árbol del escenario se aplana en orden DFS y cada hoja recibe un `target_index` secuencial. Los nodos lógicos (`and`, `or`, `not`) no se persisten como filas, aportan estructura solo en memoria. El reenlace entre la hoja del modelo Pydantic de la Capa 3 y su fila es posicional por `target_index` en ese mismo orden DFS, lo que evita guardar un camino jerárquico en la base de datos. Sus columnas son `id`, `run_id`, `target_index`, `target_type`, `description`, `matched`, `matched_at`, `trigger_event` y `reset_at`. Este último permite reintentos sin borrar el historial, ya que el intento actual es el conjunto de filas con `reset_at` nulo. Tiene índice ordinario sobre `run_id` para recuperar eficientemente todos los resultados de un run.

RunEvent guarda el flujo completo de eventos del monitor asociados a un run (`monitor_type`, `event_type`, `details_json`, `timestamp`). Sirve como bitácora auditable del ejercicio, independiente de qué targets matcheen o no: permite reconstruir a posteriori la secuencia de acciones del estudiante. Tiene índice ordinario sobre `run_id`.

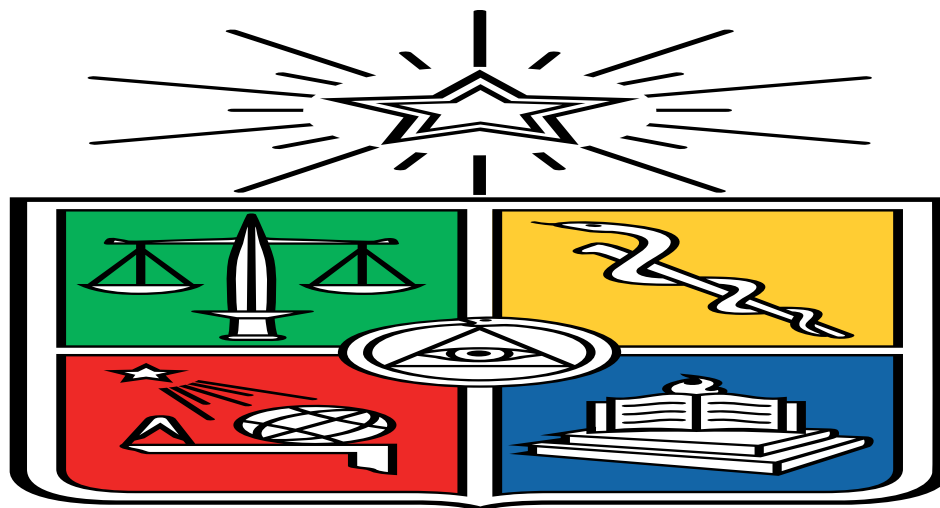


Figura 3.3: [Placeholder] Diagrama entidad-relación del modelo de datos de FlexiPwn.

3.4. Aislamiento de Entornos

El aislamiento entre los entornos de práctica de distintos participantes opera en dos dimensiones complementarias: aislamiento de red y aislamiento de filesystem.

Modelo de red de doble capa

Cada entorno instancia dos redes Docker independientes con roles diferenciados. La red interna (`flexipwn-{env_id}`), creada con `internal=True`, conecta el sistema vulnerable y el sistema atacante entre sí sin permitir tráfico hacia el exterior. Esta configuración garantiza que un ejercicio de explotación activa no genere tráfico hacia redes externas ni hacia otros entornos del sistema. La red externa (`flexipwn-{env_id}-ext`), un bridge estándar sin restricciones, conecta exclusivamente el contenedor atacante al host. Esto permite que el participante acceda al entorno atacante mediante SSH desde su máquina, manteniendo al mismo tiempo el contenedor vulnerable completamente aislado del exterior.

El contenedor atacante opera en ambas redes simultáneamente: su interfaz en la red interna le permite resolver el nombre DNS del vulnerable (`flexipwn-{env_id}-vulnerable`) y establecer conexiones hacia él. Su interfaz en la red externa permite que los port bindings definidos en el YAML del escenario sean alcanzables desde el host. El contenedor vulnerable solo tiene presencia en la red interna.

El contenedor *sidecar* de captura de red (Sección 4.4.4) no se conecta a ninguna red Docker directamente: comparte el namespace de red del contenedor vulnerable mediante `--network container:{nombre_vulnerable}`, lo que le permite observar todo el tráfico del vulnerable sin introducir interfaces adicionales en la topología de red del entorno.

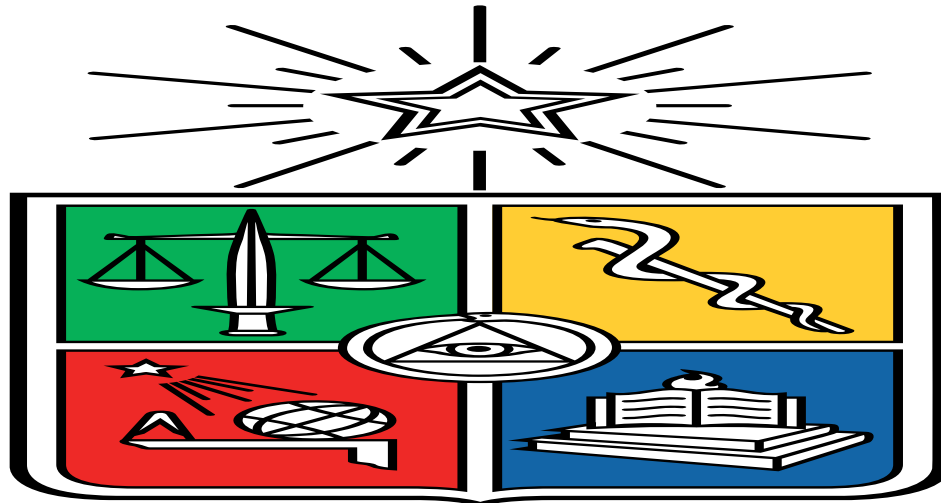


Figura 3.4: [Placeholder] Topología de red del entorno: red interna y externa con el *sidecar* y los tres contenedores. El vulnerable solo en la red interna, el atacante en ambas, y el *sidecar* compartiendo el *netns* del vulnerable.

Aislamiento de filesystem

Los bind mounts efímeros descritos en la Sección 4 se crean en su mayoría con permisos 0o700, accesibles únicamente por el proceso del daemon rootless. La base de volúmenes y el directorio de captura de red usan ese modo. Una excepción son los directorios de logs que se montan al contenedor vulnerable, creados con 0o777 para que el proceso del contenedor pueda escribir sus archivos de log. Al destruir el entorno, el árbol de directorios completo se elimina mediante `shutil.rmtree()`. El contenedor *sidecar* de captura se detiene y elimina al inicio del ciclo de destrucción, antes de detener el contenedor vulnerable y atacante, y su directorio de captura se borra en ese mismo paso.

Capítulo 4

Implementación

Este capítulo describe las decisiones técnicas tomadas durante la implementación de FlexiPwn, organizadas según las capas de la arquitectura.

4.1. Stack Tecnológico

Las decisiones de *stack* tecnológico de FlexiPwn fueron tomadas en base a un contexto particular: el sistema debe ser autohospedable en hardware de recursos limitados, y suficientemente simple para que un educador no tenga que invertir demasiado tiempo configurando la infraestructura del sistema, para que pueda operar y extender los escenarios de forma rápida. Esta sección describe los componentes seleccionados, organizados según las cuatro capas de la arquitectura. La justificación profunda de la tecnología de virtualización se desarrolla por separado en la Sección 4.2.

4.1.1. Lenguaje y gestión de dependencias

Python 3.12 es el lenguaje principal. La elección se sostuvo en que era el lenguaje que más se dominaba y que coincidía con tener un gran ecosistema de herramientas de infraestructura y orquestación, y con ser de los lenguajes más utilizados en ciberseguridad [17].

La gestión de dependencias y entornos virtuales se realiza con `uv` [18], que reemplaza la combinación tradicional de `pip` y `venv` que agiliza y unifica instalación, sincronización y ejecución de scripts en una sola herramienta.

4.1.2. Capa 1: Virtualización de Entornos

Los entornos vulnerables y atacantes se ejecutan como contenedores Docker en modo *rootless*, gestionados desde Python con el SDK oficial `docker-py`. Se decidió utilizar el modo

rootless para minimizar la superficie de ataque del sistema: al no requerir privilegios de root ni un *daemon* centralizado, se reduce el riesgo de que una vulnerabilidad en el sistema de contenerización comprometa la máquina anfitriona. Esta decisión es un pilar de la seguridad central del proyecto y se justifica en detalle en la Sección 4.2. Toda la interacción con Docker se encapsula tras la interfaz abstracta `EnvironmentProvider` (ver capítulo 3), de modo que el resto del sistema no depende de la tecnología de virtualización o contenerización concreta.

4.1.3. Capa 2: Monitoreo Pasivo

La Capa 2 no usa librerías de monitoreo externas: observa el estado de los contenedores exclusivamente a través del SDK de Docker y de lectura de archivos en el host. El monitor de **filesystem** usa `container.diff()`; el de **procesos**, `container.top()`; el de **logs** lee directamente los archivos que el contenedor escribe en *bind mounts*; y el de **red** parsea la salida de un contenedor *sidecar* basado en la imagen `nicolaka/netshoot` que ejecuta `tcpdump`. Esta implementación se describe en detalle más adelante en este capítulo.

4.1.4. Capa 3: Motor de Evaluación

Los escenarios se describen en archivos YAML parseados con PyYAML y validados con Pydantic v2. La separación entre *parsing* (PyYAML) y validación (Pydantic) fue intencional: los errores de configuración se detectan con mensajes claros al cargar el escenario, no durante la ejecución. Pydantic v2 fue elegido por su integración nativa con SQLAlchemy y por el rendimiento de su núcleo en Rust.

El motor de evaluación de condiciones de éxito es código Python propio, sin dependencias externas. Los *frameworks* de reglas orientados a detección en producción (como Sigma o Yara) tienen una expresividad que excede los requerimientos de FlexiPwn y una curva de aprendizaje que complicaría la creación de escenarios. El motor implementa los tipos de target atómicos y los nodos lógicos `and/or/not` descritos en el capítulo 3, y es completamente independiente de la infraestructura de contenedores: sus pruebas corren en memoria, sin necesidad de Docker.

4.1.5. Capa 4: Administración

La interfaz para educadores y estudiantes es una CLI construida con Typer, que genera comandos, subcomandos, validación de argumentos y páginas de ayuda a partir de las anotaciones de tipo de Python. Rich complementa a Typer con el *output* visual: tablas de escenarios, paneles de progreso e indicadores de estado. Sobre esta base, la administración opera como un *daemon* con un REPL interactivo construido con `prompt_toolkit`, que permite al educador operar el sistema en *foreground* o conectarse a un *daemon* en segundo plano. Este componente se detalla en la sección 4.6.

El estado del sistema (escenarios, participantes, *runs* y resultados) se persiste en SQLite a

través de `SQLModel`, que combina `SQLAlchemy` como motor de base de datos con `Pydantic v2` como sistema de validación, permitiendo definir los modelos una sola vez. `SQLite` es apropiado para la escala de `FlexiPwn` (decenas a centenares de participantes) y reduce el despliegue a un único directorio con la plataforma y el archivo `.db`. Las contraseñas de los participantes se almacenan como *hash* `bcrypt`.

4.1.6. Testing

El *framework* de pruebas es `pytest`, con `pytest-mock` para los dobles de prueba. Los tests del motor de evaluación y de los *schemas* corren en memoria, sin `Docker`, lo que mantiene un ciclo de desarrollo ágil. Los tests de integración que requieren contenedores reales se marcan con `pytest.mark.integration` para ejecutarlos de forma opcional cuando el entorno tiene `Docker` disponible.

4.1.7. Resumen del stack

La Tabla 4.1 consolida los componentes con su versión mínima de referencia (según `pyproject.toml`), el rol que cumplen y la capa a la que pertenecen.

Tabla 4.1: Componentes del *stack* tecnológico de `FlexiPwn`.

Componente	Rol en <code>FlexiPwn</code>	Capa
Python ≥ 3.12	Lenguaje principal	Transversal
uv	Gestor de dependencias y entornos	Transversal
docker ≥ 7.0	SDK Python para el ciclo de vida de contenedores	Capa 1
pyyaml ≥ 6.0	<i>Parsing</i> de los <code>YAML</code> de escenarios	Capa 3
pydantic ≥ 2.0	Validación del <i>schema</i> de escenarios	Capa 3
typer ≥ 0.15	<i>Framework</i> de la CLI	Capa 4
rich ≥ 13.0	Tablas, paneles y <i>output</i> con color	Capa 4
prompt_toolkit ≥ 3.0	REPL interactivo del <i>daemon</i>	Capa 4
sqlmodel $\geq 0.0.14$	ORM sobre <code>SQLite</code> (persistencia)	Capa 4
bcrypt ≥ 4.0	<i>Hash</i> de contraseñas de participantes	Capa 4
pytest ≥ 8.0	<i>Testing</i> (con <code>pytest-mock</code>)	Transversal

El monitoreo de la Capa 2 no aparece en la tabla como dependencia externa porque se implementa enteramente sobre el SDK de `Docker` (`container.diff()`, `container.top()`) y lectura de archivos, sin librerías de observación adicionales.

4.1.8. Consideraciones de evolución

Dos decisiones del *stack* están marcadas para revisión futura. La primera es migrar el `EnvironmentProvider` de `Docker rootless` a **Podman rootless**, que al ser *daemonless* elimina el *daemon* centralizado como superficie de ataque, y que se podría lograr gracias a que

la interfaz abstracta hace que este cambio no afecte a las capas superiores. La segunda es complementar la CLI con una **interfaz web** para facilitar la administración del sistema y poder exponer de forma más visual el progreso de los estudiantes.

4.2. Decisión de Tecnología de Virtualización

FlexiPwn necesita instanciar entornos de práctica vulnerables para múltiples estudiantes en paralelo. Cada entorno debe ser aislado, reseteable y monitoreable desde Python. La elección de tecnología de virtualización impacta directamente en cinco dimensiones que se ponderaron según su importancia para el proyecto: seguridad del *host* (30 %), por tratarse de ejercicios con exploits activos; rapidez de inicialización (20 %), que define la experiencia del estudiante al pedir su entorno; simplicidad de implementación (20 %), que afecta la velocidad de desarrollo y la mantenibilidad; monitoreo programático (15 %), es decir, la facilidad para inspeccionar, por ejemplo, el *filesystem* y los logs desde Python; y portabilidad (15 %), para que el sistema funcione tanto en equipos educativos como en los servidores del laboratorio CLCERT.

Se evaluaron cuatro alternativas: Docker *rootful*, Docker *rootless*, Podman *rootless* y KVM/libvirt (máquinas virtuales completas). Se excluyeron gVisor y Kata Containers del análisis principal porque, si bien son opciones válidas, añaden una complejidad operativa que escapa el alcance educativo del proyecto en su fase actual.

4.2.1. Alternativas evaluadas

Docker *rootful* es la configuración por defecto de Docker Engine: un *daemon* con privilegios de root en el *host*. Su SDK para Python (**docker-py**) es maduro y muy documentado, pero su modelo de seguridad es inadecuado para ejercicios de ataque. Tectonic, la plataforma de *cyber range*, advierte explícitamente que en modo privilegiado un usuario que obtenga root dentro de un contenedor puede obtener root en el *host* [15]. Este riesgo no es teórico: las vulnerabilidades CVE-2025-31133, CVE-2025-52565 y CVE-2025-52881 en **runc** (noviembre de 2025) permiten escape de contenedor y root en el *host* mediante manipulación de *symlinks*, y el patrón se repite históricamente [4].

Docker *rootless* ejecuta tanto el *daemon* como los contenedores dentro de un *user namespace*. Si un atacante escapa del contenedor, solo obtiene los permisos del usuario de *host* que ejecuta el *daemon*, nunca root [19]. El SDK de Python es idéntico al de *rootful*.

Podman *rootless* tiene una arquitectura *daemonless*: cada comando crea un proceso hijo sin un *daemon* central con privilegios, lo que elimina la superficie de ataque que representa un *daemon* compartido. Es *rootless* por diseño desde su origen y expone una API compatible con la de Docker, utilizable mediante **docker-py** o el cliente nativo **podman-py** [20, 21].

KVM/libvirt ofrece el mayor aislamiento posible, un kernel separado por máquina virtual, pero con un costo prohibitivo para el caso de uso: tanto el arranque de cada instancia

como su consumo de memoria son significativamente mayores que los de un contenedor [22], y la operación a escala suele requerir herramientas como Terraform, Packer y Ansible.

4.2.2. Comparación ponderada

La Tabla 4.2 resume el puntaje de cada alternativa en una escala de 1 a 5 por criterio, multiplicado por el peso respectivo. Los valores son cualitativos transformados a escala ordinal: representan la posición relativa de cada opción en el contexto específico de FlexiPwn (educación, escala pequeña a mediana, monorepo Python, despliegue autohospedable), no mediciones de *benchmark*.

Tabla 4.2: Comparación ponderada de tecnologías de virtualización.

Criterio (peso)	D. rootful	D. rootless	Podman	KVM
Seguridad (30 %)	1 / 0.30	4 / 1.20	5 / 1.50	5 / 1.50
Startup (20 %)	5 / 1.00	4 / 0.80	4 / 0.80	2 / 0.40
Simplicidad (20 %)	5 / 1.00	4 / 0.80	4 / 0.80	2 / 0.40
Monitoreo (15 %)	5 / 0.75	5 / 0.75	5 / 0.75	3 / 0.45
Portabilidad (15 %)	5 / 0.75	4 / 0.60	4 / 0.60	2 / 0.30
Total	3.80	4.15	4.45	3.05

4.2.3. Decisión: Docker *rootless* con interfaz abstracta

FlexiPwn utiliza Docker *rootless* como tecnología de virtualización principal, expuesto a través de la interfaz abstracta `EnvironmentProvider` descrita en el Capítulo 3. La decisión se apoyó en los tres criterios de mayor peso. En **seguridad**, el modo *rootless* elimina el vector de escape hacia root en el *host*, que es el riesgo más crítico en una plataforma con ejercicios de *privilege escalation*. KVM ofrecería un aislamiento mayor, pero la diferencia es marginal en un entorno educativo controlado donde los participantes son estudiantes y no atacantes externos adversariales. En **rapidez de inicialización**, los contenedores arrancan en menos de un segundo con la imagen precargada, frente a los 5 a 30 segundos de una VM, diferencia decisiva cuando 30 o más estudiantes crean sus entornos al inicio de una clase. En **simplicidad**, `docker-py` es la API de orquestación de contenedores más madura y documentada disponible, lo que acelera el desarrollo dentro de los plazos del proyecto.

Hay que notar que Podman *rootless* obtuvo el puntaje ponderado más alto (4.45 frente a 4.15) y es arquitectónicamente más sólido al ser *daemonless*. La razón para no elegirlo como *backend* principal fue pragmática: `podman-py` es funcional pero aún en desarrollo activo, con diferencias sutiles respecto a `docker-py` en operaciones de *streaming* de logs y *stats*, y la documentación y comunidad de `docker-py` es considerablemente mayor. En términos prácticos, ambas alternativas ofrecen garantías de seguridad equivalentes para los ejercicios de FlexiPwn. La diferencia arquitectónica es significativa en producción a gran escala, pero marginal para una instalación educativa de algunas decenas de usuarios concurrentes. La interfaz abstracta permite incorporar Podman como *backend* alternativo en una iteración futura sin modificar las capas superiores.

4.2.4. Trabajo futuro en aislamiento

Existen tres líneas de evolución posibles, donde las últimas dos apuntan a vulnerabilidades más sofisticadas. La primera es implementar un `PodmanProvider`, mejora natural en seguridad arquitectónica por su base *rootless*. La segunda es usar gVisor (`runsc`) como *runtime* alternativo para ejercicios con vulnerabilidades más delicadas: añade un kernel en espacio de usuario que intercepta *syscalls* con un *overhead* mucho menor que una VM, aunque implementa solo una parte de las *syscalls* de Linux, lo que puede causar incompatibilidades con ciertos exploits. La tercera es Kata Containers para ejercicios de *kernel exploitation*, donde Docker *rootless* no basta porque el kernel del *host* sigue siendo compartido. Kata ejecuta los *workloads* en una VM ligera con su propio kernel, resolviendo ese límite a cambio de un arranque algo mayor [23].

4.3. Capa 1: Orquestador de Contenedores

La Capa 1 está implementada por la clase `DockerRootlessProvider`, la única aplicación concreta de la interfaz abstracta `EnvironmentProvider` hasta el momento. Este *provider* encapsula todas las llamadas al SDK de Docker (`docker-py`) y materializa cada `Environment` como un conjunto de recursos Docker (redes, contenedores y *bind mounts*) con identificación y etiquetado uniforme, de modo que las capas superiores puedan operar entornos sin conocer detalles del *runtime*. Esta sección describe los aspectos de implementación y presenta las justificaciones que la llevaron a su estado actual.

4.3.1. Detección del socket Docker

`DockerRootlessProvider` resuelve la ubicación del socket Docker al construirse, sin requerir configuración explícita en la mayoría de los casos. Si la configuración fija un `docker_socket` explícito, se usa esa ruta y la detección automática no se ejecuta. En su ausencia, la detección recorre cuatro candidatos en orden de prioridad: primero respeta la variable de entorno `DOCKER_HOST` si está definida, permitiendo que el educador apunte a un socket personalizado; de no presentarla, busca `$XDG_RUNTIME_DIR/docker.sock`, ubicación estándar en sistemas Linux modernos; luego `/run/user/{uid}/docker.sock`, la ruta por defecto de la instalación *rootless* de Docker; y por último `~/.docker/run/docker.sock`, el socket de Docker Desktop en macOS, que habilita el desarrollo en esa plataforma. Si ninguno existe, se levanta un `SocketNotFoundError` con un mensaje que indica al usuario las rutas que se intentaron, evitando errores opacos del SDK.

4.3.2. Identificación y etiquetado de recursos

Cada entorno recibe un identificador único `env_id` de la forma `run-XXXXXXXX` (el prefijo “run” y ocho caracteres hexadecimales derivados de un UUID), del que se derivan los nombres de otros recursos asociados: `flexipwn-{env_id}-vulnerable`

y `flexipwn-{env_id}-attacker` para los contenedores, y `flexipwn-{env_id}` y `flexipwn-{env_id}-ext` para las redes interna y externa. Esta convención permite recuperar cualquier recurso a partir del `env_id` persistido en la base de datos sin mantener un mapeo aparte.

Todos los recursos creados por la plataforma se etiquetan con `flexipwn.managed=true` más un conjunto de etiquetas contextuales `flexipwn.env_id`, `flexipwn.scenario_id` y `flexipwn.participant_id`. Estas etiquetas son la base de la operación de limpieza `cleanup_all`, que recorre los recursos del *daemon* filtrando por la etiqueta principal y elimina cualquier entorno huérfano que haya sobrevivido a una caída del proceso administrador, permitiendo recuperar la consistencia entre el estado persistido en SQLite y el estado real de Docker sin afectar recursos ajenos a FlexiPwn.

4.3.3. Creación con *rollback*

La operación `create()` levanta de forma sucesiva el directorio base de *bind mounts*, las redes interna y externa, el contenedor vulnerable, el contenedor atacante y, cuando el escenario lo requiere, el *sidecar* de captura de red. Un fallo a mitad de camino dejaría recursos huérfanos en Docker y archivos sin uso en el *host*, lo que con el tiempo contaminaría el sistema y dificultaría el diagnóstico de posibles errores.

Para evitarlo, el método mantiene un conjunto de banderas booleanas (`created_dirs`, `created_network`, `created_vulnerable` y `created_sniffer`) que se activan a medida que los pasos completan con éxito. Cualquier excepción captura el flujo en un único bloque `except` que invoca `_rollback`, encargado de deshacer lo que estaba creado en el momento del fallo. En definitiva, `create()` puede terminar de dos formas: o retorna un `Environment` completamente funcional, o lanza la excepción original con todos los recursos parciales eliminados.

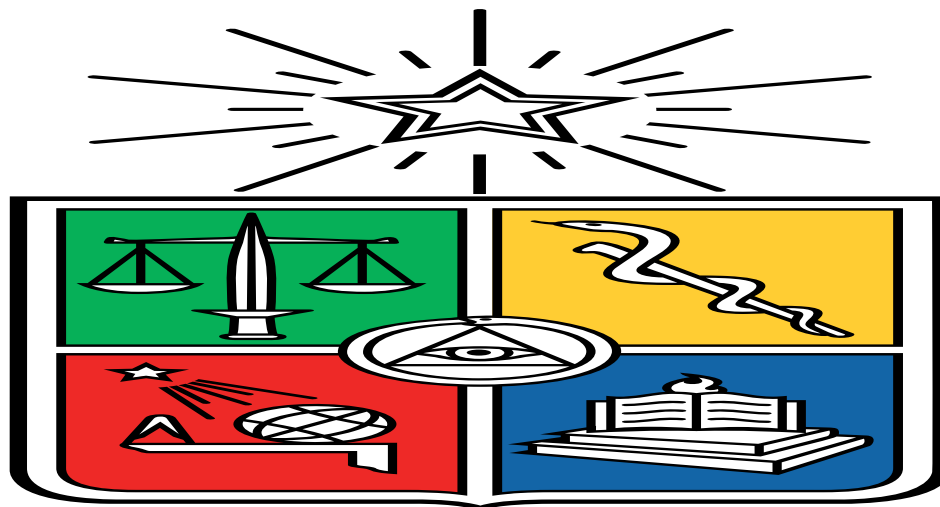


Figura 4.1: [Placeholder] Máquina de estados de un *run*: `pending`, `running`, `stopping`, `resetting`, `completed`, `failed`, `timeout` y `stopped`.

4.3.4. Estrategia de *baseline* para los monitores

Los monitores de Capa 2 necesitan distinguir entre los cambios que el contenedor produce durante su inicialización (instalación de paquetes, arranque de servicios, creación de archivos temporales) y los cambios provocados por el estudiante durante el ejercicio. Para esto, el *provider* captura un *baseline* del estado del contenedor una vez que su inicialización se considera concluida.

La estrategia de detección de ese momento es híbrida y se elige automáticamente según las capacidades de la imagen. Si el contenedor vulnerable declara un **HEALTHCHECK**, el *provider* consulta periódicamente el estado de salud reportado por Docker hasta que alcance **healthy**. Esta forma es la preferida porque elimina la incertidumbre de un tiempo de espera fijo y se ajusta al tiempo real de arranque de cada imagen. Si la imagen no define un **HEALTHCHECK**, el *provider* recurre al modo **delay**: espera un intervalo configurable (con valor por defecto y ajustable por escenario mediante **startup_delay_seconds**) antes de tomar el *baseline*. Si el **HEALTHCHECK** existe pero no llega a **healthy** dentro del *timeout* configurado, el *provider* toma el *baseline* de todas formas y emite una advertencia, priorizando la disponibilidad del entorno por sobre la garantía de arranque limpio.

El *baseline* resultante se guarda en disco, junto con la estrategia que se usó para capturarlo. Esto permite que el *provider* se reinicie sin perder la referencia, evitando que reinicios del *daemon* marquen como cambios del estudiante todo lo que el contenedor escribió durante su arranque original.

4.4. Capa 2: Monitoreo Pasivo

La Capa 2 está compuesta por cuatro monitores independientes: **FilesystemMonitor**, **ProcessMonitor**, **LogMonitor** y **NetworkMonitor**, y que a su vez están coordinados por un **MonitorOrchestrator** por entorno. El orquestador es responsable de invocar el ciclo de *polling* de cada monitor con intervalos configurables, manejar el *timeout* global del escenario, lanzar la señal de detención cuando un contenedor desaparece y emitir todos los eventos generados a través del único **callback on_event** que consume la Capa 3. Los tres primeros monitores observan el contenedor vulnerable exclusivamente desde fuera, usando el SDK de Docker o lectura directa de archivos del *host*, mientras que el cuarto se apoya en un *sidecar* de captura y que se describe en la Sección 4.4.4.

4.4.1. FilesystemMonitor

El monitor de *filesystem* consulta **container.diff()** en cada iteración a través del **DockerRootlessProvider**. La llamada retorna el conjunto de archivos creados, modificados o eliminados respecto a la imagen base del contenedor. El monitor compara ese conjunto contra los *paths* que ya emitió en iteraciones anteriores (**_seen_paths**) y genera un **MonitorEvent** por cada *path* nuevo o cuya naturaleza haya cambiado (por ejemplo, un archivo modificado que pasa a borrado). Los **event_type** emitidos son **file_created**, **file_modified** y

`file_deleted`, según el código que Docker asigna a cada diferencia.

4.4.2. ProcessMonitor

El monitor de procesos consulta `container.top()`, también invocado a través del `DockerRootlessProvider`, y reconstruye en memoria el árbol de procesos del contenedor. El *baseline* de procesos se toma en la primera invocación tras el arranque, una vez que el *provider* ya garantizó que el contenedor está estable (Sección 4.3.4). En cada iteración posterior se identifican los procesos nuevos por un `process_id` derivado de combinar el `pid` con el tiempo de inicio reportado por `lstart`, lo que evita falsos negativos cuando el sistema reutiliza un `pid` liberado.

Los eventos `process_spawned` incluyen no solo el comando del proceso (`cmd`) sino también los comandos de su proceso padre (`ppid_cmd`) y de la cadena completa de ancestros (`ancestor_cmds`), reconstruidos desde `container.top()`. Esta información viaja en el evento y es la Capa 3 la que decide si filtrar por *euid*, por substrings en `cmd`, en `ppid_cmd` o en cualquiera de los ancestros. La Capa 2 se mantiene agnóstica respecto a qué procesos son interesantes, preservando la separación entre observación y evaluación.

4.4.3. LogMonitor

El monitor de *logs* no usa el SDK de Docker: lee los archivos del *host* en las rutas que el contenedor escribe a través de *bind mounts* declarados en el campo `log_paths` del escenario. En la primera invocación posiciona el cursor al final de cada archivo para no emitir eventos por el contenido histórico anterior al *run*, y en las invocaciones siguientes lee solo las líneas nuevas desde la última posición conocida.

Cada línea se intenta parsear como JSON. Si el *parsing* tiene éxito, el evento `log_entry` expone los campos estructurados en `details["parsed"]`. Si la línea es texto plano (caso de logs como el `general_log` de MySQL), el evento la expone en `details["raw_line"]`. Esta auto-detección por línea evita que el educador deba declarar el formato del *log* en el escenario y permite mezclar fuentes estructuradas y no estructuradas en el mismo ejercicio. El monitor también detecta el truncamiento del archivo, por ejemplo, cuando la aplicación reinicia y vuelve a abrir su *log*, comparando el tamaño actual contra el último observado. Si el archivo decreció, reposiciona el cursor al inicio y recomienza la lectura.

4.4.4. NetworkMonitor

Para escenarios que requieren observar tráfico de red, la Capa 2 instancia un cuarto monitor: `NetworkMonitor`. A diferencia de los tres monitores anteriores, este monitor depende de un contenedor *sidecar* que comparte el namespace de red del sistema vulnerable.

Contenedor *sidecar*

Al crear el entorno, si el árbol de targets del escenario contiene al menos una hoja `network_*` (sea como target de primer nivel o anidada en cualquier nivel dentro de nodos `and`, `or` o `not`), `DockerRootlessProvider` lanza un contenedor adicional basado en la imagen `nicolaka/netshoot` con la siguiente configuración:

- **Namespace de red compartido:** mediante `--network container:{vulnerable}`, el *sidecar* comparte exactamente el mismo namespace de red que el vulnerable. Esto hace visible todo el tráfico del vulnerable desde dentro del *sidecar*, incluyendo comunicaciones por la interfaz de loopback (127.0.0.1), que de otro modo no serían capturables desde una perspectiva externa.
- **Comando de captura:** `tcpdump -i any -A -n -l`, con un filtro opcional definido en el campo `environment.capture_filter` del YAML del escenario (por ejemplo, `port 3306` para restringir la captura a tráfico MySQL).
- **Salida en bind mount:** la captura se escribe en un archivo `traffic.txt` en un directorio del host con permisos `0o700`, al que solo tiene acceso el proceso de FlexiPwn.

El binario `tcpdump` no se ejecuta desde su ruta original, sino que se copia a `/tmp/fpcap` y se lanza desde ahí. El motivo es un *workaround* de AppArmor: el perfil del host asociado a la ruta estándar del binario deniega que el *sidecar* reciba las señales de `rootlesskit`, lo que dejaría al *sniffer* imposible de detener en el *teardown*. Al ejecutarlo desde una ruta que no coincide con el perfil, el proceso corre sin limitaciones y la destrucción del entorno puede terminarlo de forma limpia.

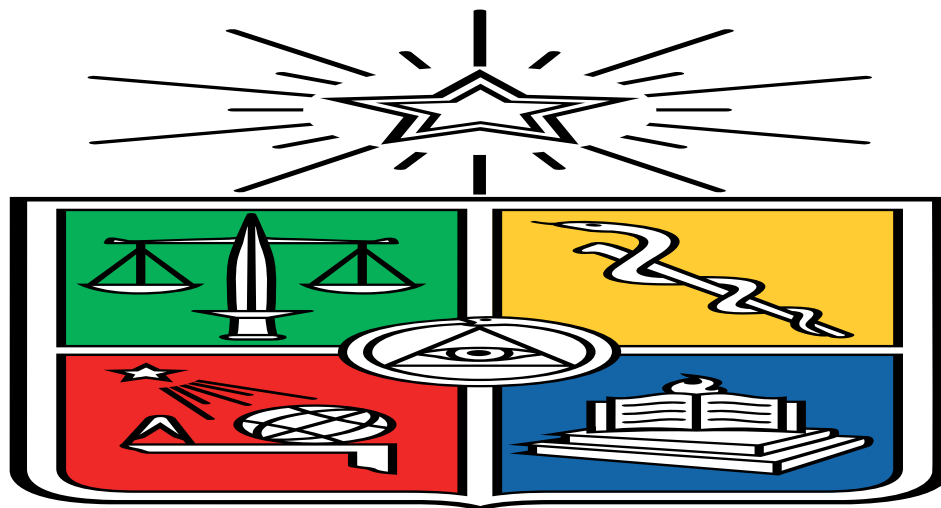


Figura 4.2: [Placeholder] Pipeline de captura de red: `tcpdump` escribe `traffic.txt` en un *bind mount* desde el *netns* del vulnerable, y el *parser* emite `network_payload` y `network_connection`.

El uso de `-n` desactiva la resolución de nombres, forzando IPs numéricas en el output y evitando que el parser del monitor falle ante hostnames como `localhost`. El flag `-l` acti-

va el line-buffering de `tcpdump`, reduciendo la latencia entre la escritura del paquete y su disponibilidad en el archivo del host.

Parsing packet-based

`NetworkMonitor` lee el archivo `traffic.txt` de forma incremental en cada ciclo de polling, procesando el output de `tcpdump` como unidades de paquete completo en lugar de línea por línea. Esta decisión fue necesaria porque el output de `tcpdump -A` en modo `-i any` mezcla bytes ASCII del payload con bytes binarios de los headers TCP/IP, y el carácter `0x0a` (LF) puede aparecer dentro de los headers, partiendo artificialmente el payload en múltiples líneas si se procesa con `splitlines()`.

El monitor detecta las cabeceras de paquete mediante una expresión regular que acepta el formato de `-i any` (que incluye el nombre de interfaz y dirección entre el timestamp y la IP) y extrae de ellas las IPs y puertos de origen y destino. Las líneas entre cabeceras se acumulan en un buffer de payload. Cuando se detecta una nueva cabecera, el buffer del paquete anterior se procesa: se extrae todo el texto en rango ASCII imprimible (`0x20--0x7e`), y si el resultado supera los cuatro caracteres se emite un `MonitorEvent` de tipo `network_payload` con el texto extraído y los metadatos del paquete. Adicionalmente, si la cabecera contiene `Flags [S.]`, se emite un evento `network_connection` con la IP y puerto de destino.

Una implicación importante del modelo packet-based es que el último paquete de una ráfaga de tráfico corta podría quedar atrapado en el buffer si no llega una cabecera posterior que dispare su procesamiento. Para evitar esto, al final de cada ciclo de polling el monitor procesa el paquete en curso si el buffer no está vacío, sin esperar al siguiente ciclo.

4.4.5. Limitaciones del Monitoreo por Polling

El monitor de *filesystem* usa `container.diff()`, que retorna una *snapshot* del estado acumulado del *filesystem* desde el inicio del contenedor. Para distinguir los cambios producidos a iniciar el sistema de los cambios realizados por el estudiante, se captura un *baseline* del *diff* una vez que el contenedor está estable: si la imagen tiene un `HEALTHCHECK` configurado, el sistema espera hasta que el contenedor reporte estado `healthy`, de lo contrario, espera un *delay* configurable (por defecto 3 segundos) como mecanismo de respaldo. Esto crea una ventana de tolerancia al inicio del *run* en la que cambios reales del estudiante podrían absorberse en el *baseline* si ocurren antes de que el contenedor se considere estable.

De forma análoga, el monitor de procesos consulta `container.top()` cada dos segundos. Procesos que nacen y terminan dentro de ese intervalo son invisibles para el sistema. Los eventos no detectables en el estado actual incluyen: procesos efímeros con duración menor a dos segundos, técnicas de *PPID spoofing*, y ataques *fileless* mediante llamadas al sistema como `ptrace` o `memfd_create`. Para el alcance educativo del proyecto, este nivel de detección es aceptable dado que los ejercicios no requieren detectar procesos sub-segundo ni técnicas de evasión avanzadas esperables en un atacante universitario.

El monitor de *logs* hereda dos limitaciones del seguimiento por posición de cursor. La primera es la rotación: si la aplicación rota su archivo (*rename* del antiguo, creación de uno nuevo en la misma ruta), el monitor sigue leyendo el original y deja de ver las líneas que se escriben en el archivo nuevo hasta el siguiente ciclo, donde reabre por ruta y vuelve a posicionar el cursor al final. La segunda es la dependencia del *flush* de la aplicación: las líneas que quedan en *buffer* del lado del contenedor, solo se vuelven visibles para el *host* cuando la aplicación las descarga, no en tiempo real. Para los escenarios actuales esto es aceptable porque las aplicaciones afectadas configuran *logging* con *flush* inmediato, pero un escenario con una aplicación menos cooperativa exigiría forzar el *flush* desde el *Dockerfile* de la imagen vulnerable.

El monitor de red comparte el principio de pasividad pero suma sus propias restricciones. Por construcción, solo extrae el contenido ASCII imprimible del *payload*, así que protocolos binarios o cifrados (TLS, el protocolo binario de MySQL en ciertas configuraciones) exponen únicamente sus metadatos, como IPs, puertos y *flags* TCP, no su contenido legible. Las reglas *network_payload* no matchean dentro de un canal TLS sin descifrarlo, lo que excluye los escenarios web sobre HTTPS sin haber manipulado TLS dentro del entorno. La segunda restricción es la fragmentación: un *payload* grande (por ejemplo, una consulta SQL larga) puede distribuirse entre varios paquetes TCP, y como el *parser* opera por unidad de paquete completo, una expresión regular cuyo *match* cruce ese límite no se dispara. En el contexto educativo, los *payloads* diseñados para los escenarios actuales caben en un único paquete. Un escenario con tráfico de mayor volumen requeriría reensamblado a nivel de flujo TCP, lo que excede el alcance de la implementación actual.

4.5. Capa 3: Motor de Evaluación

Los tipos de target implementados en el estado actual del sistema son:

file_created, file_modified Operan sobre eventos del monitor de filesystem. Permiten detectar creación o modificación de archivos en rutas absolutas o mediante patrones glob.

process_running Opera sobre eventos del monitor de procesos. Detecta procesos activos por effective UID y substring en el comando.

log_pattern Opera sobre eventos del monitor de logs. Aplica un diccionario de campo→regex sobre los campos de la entrada de log. Si la línea se parseó como JSON, los campos se buscan en la entrada estructurada. En cambio, si el campo declarado es *raw_line*, la regex se aplica sobre el texto plano de la línea. Este segundo modo es el que usan los escenarios MySQL, cuyo *general_log* no es JSON.

network_payload Opera sobre eventos *network_payload* del *NetworkMonitor*. Aplica un diccionario de campo→regex sobre el campo *data* del evento, que contiene el texto ASCII imprimible extraído del payload del paquete. El matching se realiza con *re.IGNORECASE*, de modo que el educador no necesita contemplar variaciones de capitalización en los patrones.

`network_connection` Opera sobre eventos `network_connection` del `NetworkMonitor`. Detecta conexiones TCP establecidas (paquete SYN-ACK) hacia un puerto de destino específico, con filtro opcional por IP de destino.

`and`, `or`, `not` Nodos lógicos recursivos que permiten componer condiciones complejas sobre cualquier combinación de targets atómicos. El campo `condition: any|all` del escenario actúa como azúcar sintáctica para listas planas de targets sin nodos lógicos.

La inspección de consultas SQL no requiere un evaluador con acceso a la base de datos: se realiza sobre el tráfico de red mediante `network_payload` con un filtro `capture_filter: "port 3306"`, que captura las consultas directamente del tráfico TCP entre la aplicación y el motor de base de datos, preservando la pasividad del monitoreo. Esto exige que el motor de base de datos del escenario opere sin TLS en la conexión *cliente-servidor*: la imagen `flexipwn/vuln-sqli-mysql` construida para los escenarios actuales desactiva TLS por configuración para mantener el payload SQL visible al *sniffer*, decisión didáctica del escenario y no del sistema. Un escenario que conserve TLS forzado expondría únicamente metadatos (IPs, puertos y *flags* TCP) al monitor de red, y obligaría al educador a apoyarse exclusivamente en `log_pattern` sobre el `general_log` para detectar el ataque.

4.6. Capa 4: Administración

La Capa 4 es la cara visible del sistema para el educador: administra el ciclo de vida de los *runs*, persiste el estado en SQLite, y provee dos formas de operar la plataforma. La primera es una CLI tradicional (`flexipwn scenario list`, `flexipwn run start`, etc.) por si se quiere hacer algún tipo de *scripting*. La segunda es un *daemon* con un REPL interactivo (`flexipwn daemon start`) que mantiene en memoria los monitores de Capa 2 y las evaluaciones de Capa 3 corriendo de forma continua sobre todos los *runs* activos en paralelo. Esta sección describe los componentes que sostienen ambos caminos.

4.6.1. CLI con Typer y Rich

Toda la línea de comandos se construye sobre `Typer`, que deriva comandos, sub-comandos, validación de argumentos y páginas de ayuda a partir de los tipos declarados en las firmas de las funciones. La CLI se organiza en cuatro sub-aplicaciones: `scenario`, `participant`, `run` y `daemon`. Todas montadas bajo la aplicación raíz `flexipwn` y con sus módulos correspondientes en `layer4/cli/`. Esta división permite extender un grupo de comandos sin afectar al resto y mantiene el código de cada sub-aplicación en un solo archivo.

El módulo `Rich` se usa exclusivamente para el *output*: tablas de escenarios y *runs*, paneles con las credenciales SSH del estudiante, e indicadores con código de color para los estados (`running`, `completed`, `failed`, `timeout`). El uso de `Typer` + `Rich` es un patrón estándar para CLI en Python y reduce significativamente el código que la plataforma debe escribir para entregar una experiencia de terminal limpia.

La CLI ofrece además un modo verboso para diagnóstico: la bandera global `--verbose` (o la variable de entorno `FLEXIPWN_LOG`) eleva el *logging* a nivel `DEBUG`, que recorre todo el flujo (creación de recursos en Capa 1, emisión de cada evento en Capa 2, decisión de *match* del motor en Capa 3 y escrituras a la base en Capa 4) desde un único punto de configuración. El nivel por defecto es silencioso, de modo que el modo verboso no afecta la operación normal ni las pruebas.

4.6.2. Modelo de *daemon*: *foreground*, *detach* y *attach*

`flexipwn daemon start` arranca el proceso *daemon*. En su modo por defecto, este queda en *foreground* y abre directamente un REPL interactivo construido sobre `prompt_toolkit`, desde el que el educador puede ejecutar los mismos sub-comandos que ofrece la CLI clásica (`scenario load`, `run start`, `participant add`, etc.) sin pagar el costo de arrancar la plataforma una vez por comando. Todas las notificaciones del *daemon* viajan por un `NotificationSink` central, que por defecto las silencia: el TTY del REPL queda limpio y la vista del progreso pasa a ser *pull* mediante los comandos `dashboard` y `feed` del propio REPL, que leen de la base de datos. Para que el educador sepa cuándo abrir las vistas, la barra inferior del REPL (`bottom_toolbar` de `prompt_toolkit`) muestra un *badge* con la cantidad de hitos nuevos desde la última lectura del feed y con la cantidad de entornos activos.

Para los momentos en que el educador no quiere mantener una terminal abierta durante toda la sesión, el modo `flexipwn daemon start --detach` lanza el *daemon* en *background*, redirige su salida a `~/.flexipwn/daemon.log` y abre un socket Unix de control en `~/.flexipwn/daemon.sock` con permisos `0o600`, así que solo el dueño del proceso puede conectarse. Desde cualquier otra terminal, `flexipwn daemon attach` establece una conexión por ese socket y abre un REPL idéntico al de *foreground*. Varios clientes pueden *attachear* simultáneamente al mismo *daemon* y todos reciben el mismo *broadcast* de eventos. Cerrar un cliente *attached* con `exit` o `Ctrl+D` no detiene el *daemon*, ya que para apagarlo de verdad existe el comando dedicado `flexipwn daemon stop`.

El protocolo del socket es simple: cada mensaje es una línea UTF-8 terminada en `\n`, prefijada por un *frame* direccional que distingue las distintas categorías de mensaje que viajan en ambos sentidos: comandos del cliente, líneas de salida del servidor, solicitudes de *input* interactivo cuando un comando dispara un *wizard*, y señales de fin de respuesta. Este *framing* permite que el cliente muestre coherentemente la salida del comando que está ejecutando, los *prompts* de los *wizards* y los eventos asíncronos que los monitores emiten en tiempo real, todos viajando sobre la misma conexión. Un proxy de consola *thread-local* se encarga de dirigir el *output* de cada comando al cliente que lo originó, evitando que los *prints* de un cliente se mezclen con los de otro cuando múltiples educadores operan en paralelo sobre el mismo *daemon*.

4.6.3. Coordinación interna: `DaemonLoop` y `SuperMonitor`

Cada *run* sigue teniendo su propio `MonitorOrchestrator` para coordinar los cuatro monitores de su entorno (Capa 2), pero el *daemon* no le da a cada uno un hilo propio:

en lugar de eso, todos los orquestadores activos se registran en un `SuperMonitor singleton` que mantiene un único hilo supervisor permanente y reparte sus invocaciones a `poll_once()` sobre un `ThreadPoolExecutor` acotado. El `ThreadPoolExecutor` viene del módulo `concurrent.futures`, estándar para tareas paralelas con paralelismo limitado: en lugar de crear un *thread* nuevo por cada invocación, operación relativamente costosa en Python, mantiene una *pool* fija de *workers* reutilizables y encola las tareas que no encuentran un *worker* libre. En cada iteración, el supervisor despacha el `poll_once()` de cada orquestador al pool y espera a que todos terminen antes de revisar *timeouts* por entorno y dormir hasta el siguiente ciclo. El intervalo entre ciclos y el tamaño del pool son configurables, pero por defecto, el supervisor itera cada dos segundos y permite hasta dieciséis `poll_once()` concurrentes. Si hay más *runs* que *workers* disponibles, los sobrantes se procesan inmediatamente a medida que los anteriores liberan su *worker*, por lo que cada ciclo despacha eventualmente a todos los *runs* activos pero limitando el consumo simultáneo de *threads* del *daemon*.

Por encima del `SuperMonitor`, el `DaemonLoop` traduce los eventos de las capas inferiores a actualizaciones de estado en la base de datos: cada `MonitorEvent` se persiste en la tabla `RunEvent` para servir como bitácora del ejercicio, y cada cambio que la Capa 3 reporta a través de `on_update` se refleja en los registros de `TargetResult` y `ExerciseRun` correspondientes. Esos mismos cambios se emiten además como `Notification` hacia el `NotificationSink`, único punto por el que el *daemon* decide entre escribir al TTY del educador o silenciar la notificación dejándola en log y, vía DB, en el *feed* persistente. Existe una *race condition* conocida entre el motor de evaluación y la observación pasiva, que el `DaemonLoop` maneja explícitamente con una regla de precedencia: los estados terminales son irreversibles. Si, por ejemplo, un contenedor desaparece inesperadamente y un *poll* en vuelo reporta `on_stopped` pero el motor ya marcó el *run* como completado, el *loop* preserva el estado terminal y no lo sobrescribe con `stopped`.

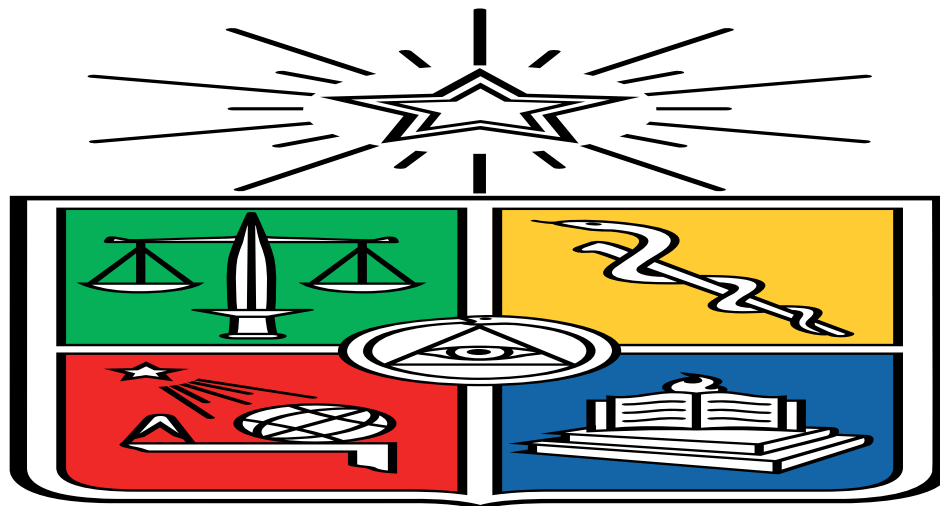


Figura 4.3: [Placeholder] Componentes del *daemon*: `DaemonLoop` y `SuperMonitor` (con su *ThreadPoolExecutor*) coordinan N `MonitorOrchestrator` y los cuatro monitores; el flujo de `MonitorEvent` pasa por el `EvaluationEngine` hacia el `NotificationSink` y la base de datos.

4.6.4. Vistas del educador: dashboard y feed

Cuando el *daemon* sostiene varios *runs* en paralelo, los hitos de progreso de cada uno mezclados sobre el cursor del REPL dejan de ser una forma útil de seguimiento. Por eso el REPL del *daemon* expone dos vistas agregadas que el educador abre bajo demanda con los comandos `dashboard` y `feed`, ambas alimentadas por la base de datos y no por el buffer en memoria del `NotificationSink`.

`dashboard` muestra una tabla con una fila por entorno activo, con *env_id*, escenario, participante, estado, objetivos cumplidos sobre el total, porcentaje y descripción del último hito con su *timestamp*. Reabrirla en cualquier momento refleja el último estado persistido, sin depender de que la sesión actual del REPL haya recibido todos los eventos. El modificador `--all` incorpora además los *runs* terminados durante el día.

`feed` es una cronología de los hitos satisfechos de todos los entornos activos, ordenados por *timestamp* y con su descripción textual. Está construida a partir del campo `matched_at` de cada `TargetResult` y de las transiciones terminales del *run*. Como las notificaciones están silenciadas por defecto en el sink, el *feed* es la vista canónica del progreso y es persistente: los hitos quedan disponibles aunque el *run* se cierre o el *daemon* se reinicie. Por defecto muestra los hitos del día, pero con `--all` recorre todo el historial almacenado.

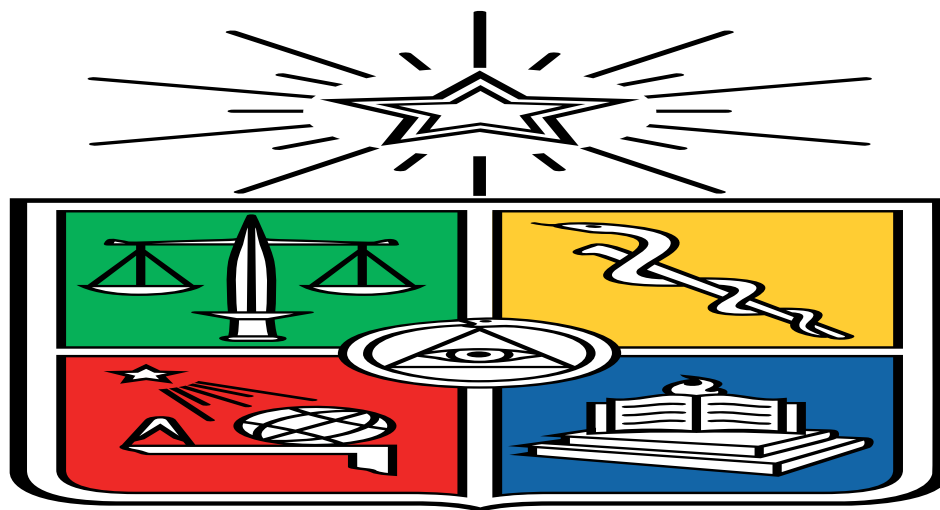


Figura 4.4: [Placeholder] Captura del dashboard, el feed y el *badge* del REPL.

4.6.5. Persistencia y ciclo de vida de un *run*

El acceso a la base de datos no se hace directamente desde la CLI ni desde el `DaemonLoop`: pasa por un módulo de repositorio (`layer4/db/repository.py`) que expone funciones de alto nivel como `create_participant`, `start_run`, `finalize_run_state`, `record_event`, `record_target_result`, entre otras, y que es el único que abre sesiones `SQLModel`. Esta separación mantiene el resto del sistema agnóstico al mapeo objeto-relacional y permite cambiar el motor de persistencia sin tocar las capas superiores.

El ciclo de un *run* es predecible. `run start` dispara un *wizard* que pide al educador escenario y participante, invoca `provider.create()` para materializar el entorno, asigna un puerto SSH del rango configurado, registra el *run* en la base con estado `running` y lo entrega al `SuperMonitor`. A partir de ese momento, los eventos de Capa 2 fluyen al motor de Capa 3 y los resultados se persisten incrementalmente. El *run* termina por una de cuatro vías: `completed` cuando todos los *targets* requeridos se satisfacen, `timeout` cuando se agota el límite de tiempo del escenario, `stopped` cuando el educador lo detiene desde la CLI o el REPL, o `failed` cuando ocurre un error irrecuperable durante la observación o la materialización. `run reset` es una operación intermedia que destruye los contenedores actuales y los recrea con un `env_id` nuevo, asociándolo al mismo escenario y al mismo participante en la base de datos: los `TargetResult` del intento previo se marcan con un `reset_at` en lugar de borrarse, lo que permite al estudiante reintentar el escenario manteniendo trazable el historial de intentos sucesivos sobre el mismo ejercicio.

El SSH del contenedor atacante no necesariamente se publica en todas las interfaces del *host*. La configuración `attacker.bind_ips` (ajustable por entorno con `FLEXIPWN_ATTACKER_BIND_IPS`) restringe la publicación del puerto SSH a una lista de IPs concretas: por ejemplo, la IP interna del laboratorio y/o la *overlay* de acceso remoto. Su valor por defecto es `None`, que publica en todas las interfaces y es cómodo para desarrollo local.

El despliegue es robusto frente a fallos parciales y *race conditions*. Antes de crear contenedores, `run start` rechaza un segundo *run* activo para el mismo par escenario-participante, evitando dejar un entorno huérfano que luego choque con la restricción de unicidad de la base. Si el entorno se materializa pero el registro del *run* en la base falla, el proveedor destruye los recursos recién creados y borra la fila pendiente, de modo que no quedan residuos. Además, `scenario validate` permite verificar que un YAML de escenario es válido sin persistirlo, separando el chequeo de la carga.

Sobre los *timestamps* que ya se persisten, sin schema adicional, `run show` y la vista `dashboard` derivan estadísticas de tiempo del *run*: la duración total y, por hito, el momento en que cada objetivo se satisfizo (campo `matched_at`) junto con el *delta* respecto al hito anterior. Los hitos se ordenan cronológicamente por `matched_at` porque el estudiante puede cumplir los objetivos en cualquier orden.

4.6.6. Credenciales de los participantes

El sistema maneja dos contraseñas con propósitos distintos para cada estudiante. La primera es la contraseña del participante: `participant add` genera un nombre de usuario del tipo `student-XXXXXX` (un prefijo fijo y seis caracteres hexadecimales derivados de `secrets.token_hex(3)`) y una contraseña en texto plano usando `secrets.token_urlsafe(12)`. La contraseña en claro se muestra al educador una única vez al crearlo y solo se persiste su *hash* `bcrypt` con *salt* aleatorio. Si el estudiante la pierde, `participant reset-password` genera una nueva, la muestra al educador y reemplaza el *hash* en la base, de modo que nunca se recupera la original.

La segunda es la contraseña de acceso SSH, que es la que el estudiante usa para conectarse

al contenedor atacante de su entorno y, desde allí, atacar al vulnerable según lo disponga el escenario. Esta contraseña es por *run*: cuando el *daemon* crea al atacante, genera una clave propia con `secrets.token_urlsafe(12)`, configura con ella la cuenta de *login* del contenedor y la persiste junto al *run*. La contraseña del participante (el *hash bcrypt*) no interviene en el flujo de acceso SSH.

4.6.7. Visor de objetivos y pistas del estudiante

Una vez dentro del contenedor atacante, el estudiante tiene acceso al comando `flexipwn`, que muestra los objetivos del escenario y sus pistas junto al estado de cada uno (cumplido o pendiente). El *daemon* es el único que conoce a la vez la descripción del escenario y el progreso de los objetivos, así que renderiza ese estado a texto plano y lo escribe en el atacante a través del contrato `exec_run` del proveedor. El comando `flexipwn` de la imagen solo lee ese archivo, sin lógica propia. El contenedor vulnerable nunca se toca: el atacante es la única sesión que la plataforma aprovisiona para el estudiante y funciona como su propio *feed*.

4.6.8. Limpieza del laboratorio

Para mantener el laboratorio operable entre clases, la capa administración ofrece comandos de limpieza. `scenario remove` elimina un escenario, con un *guard* que rechaza la operación si tiene *runs* activos asociados para no dejar entornos sin escenario de referencia. Para los reinicios completos existen `scenario reset-all` y `participant reset-all`, que borran en bloque todos los escenarios o todos los participantes respectivamente. Al ser destructivos, su confirmación no se puede saltar con una bandera: el educador debe escribir literalmente `BORRAR`.

4.6.9. Asignación masiva con batch-start

Para una sesión con varios estudiantes, iniciar los *runs* de a uno desde el REPL no escala. El comando `flexipwn run batch-start` recibe un archivo YAML con pares escenario/cantidad de *runs* a crear y los materializa de forma secuencial, uno tras otro en un bucle. El despliegue es secuencial: el asignador de puertos SSH del atacante elige el primer puerto libre con una prueba de *bind*, y dos `provider.create()` concurrentes podrían recibir el mismo puerto antes de que Docker lo ocupe, por lo que la secuencialidad es la que garantiza puertos disjuntos. El `ThreadPoolExecutor` del `SuperMonitor` solo paraleliza el *polling* de monitoreo de los entornos ya creados, no su despliegue. Opcionalmente, se permite copiar los datos a un CSV que el educador puede distribuir a los participantes, de modo que cada estudiante recibe sus credenciales y el endpoint SSH del atacante asignado a su *run*. Cada invocación crea un participante nuevo por cada entrada del YAML, así que volver a ejecutar el mismo archivo duplica los entornos en lugar de reutilizarlos.

Si la materialización de un entorno falla en mitad del *batch* (por ejemplo, la imagen no existe localmente o el puerto SSH asignado ya está tomado), el comando captura la excepción

como `ProvisionError` para ese entorno particular, sigue procesando los restantes y al final reporta los éxitos junto con la lista de entornos fallidos y la razón asociada a cada uno. En el CSV se escriben siete columnas (`scenario`, `username`, `env_id`, `ssh_port`, `ssh_password`, `status` y `error`) e incluye tanto las filas de los entornos creados (`status = created`) como las de los fallidos (`status = failed`, con la razón en `error`), para que el educador pueda decidir si reintenta esos casos puntuales sin descartar y rehacer el lote completo.

Capítulo 5

Validación

Este capítulo evalúa en qué medida FlexiPwn cumple el criterio de éxito declarado en el Capítulo 1: ofrecer validación automática, basada en el estado del sistema y sin necesidad de *flags* manuales, para ejercicios pertenecientes a varias categorías de seguridad ofensiva. La evaluación se organiza en dos planos. El primero comprueba que los escenarios funcionan de extremo a extremo: el estudiante ejecuta un *exploit* real y los objetivos correspondientes quedan marcados como cumplidos en la base de datos sin intervención del educador. Este plano corresponde exactamente al criterio de evaluación propuesto en el informe del curso de Introducción al Trabajo de Título, que comprometía únicamente verificar la capacidad del sistema para generar ejercicios con validación automática en al menos tres categorías. El segundo plano examina la robustez del sistema ante condiciones adversas y de configuración, y su uso en un entorno más realista (acceso remoto, resistencia y contención) a modo de extensión más allá de lo planteado como criterio de éxito.

5.1. Metodología y criterios de validación

El criterio del Capítulo 1 exige cubrir al menos tres categorías de ejercicio con detección automática. La validación funcional se apoya en tres escenarios incluidos en el repositorio [24], cada uno representante de una categoría distinta: escalada de privilegios (*pwning*), inyección SQL (*web*) e inyección de comandos hacia una *reverse shell* (*web*, con énfasis en detección de red). Los tres se cargan desde descripciones declarativas en YAML, se validan contra imágenes vulnerables construidas para el proyecto y disparan sus objetivos a partir de la actividad real del estudiante.

Cada escenario se valida siguiendo la misma cadena: una descripción YAML define el entorno y los objetivos; el estudiante ejecuta el *exploit* desde el contenedor atacante; los monitores de la Capa 2 emiten eventos de estado; el motor de la Capa 3 evalúa esos eventos contra el árbol de objetivos; y la Capa 4 persiste el resultado como **TargetResult** en la base de datos. El runbook de pruebas del proyecto documenta esta cadena `YAML → exploit → TargetResult` paso a paso, recorriendo los cuatro puntos observables (creación del entorno, emisión del evento, coincidencia en el motor y persistencia) con el modo verboso de depuración

activado.

En este punto conviene distinguir los dos planos de la validación. El primero, en que los escenarios funcionan y disparan sus objetivos correctamente, constituye el criterio de evaluación propuesto en el informe del curso de Introducción al Trabajo de Título, y se cubre con los tres casos descritos a continuación y con el conjunto de *tests*. El segundo, relacionado a un uso realista de un aula completa, es una extensión más allá de lo comprometido como evaluación. Se aborda parcialmente mediante pruebas unitarias y comprobaciones de la configuración de seguridad, y se reconoce explícitamente como trabajo pendiente.

5.2. Cobertura del criterio de éxito

Los tres escenarios validados cubren las tres categorías exigidas y, en conjunto, ejercitan los cuatro tipos de monitor de la Capa 2: sistema de archivos, procesos, registros (*logs*) y red. La Tabla 5.1 resume esta cobertura. El escenario de inyección SQL es el más completo en términos de evaluación lógica: combina objetivos de tipo `log_pattern` sobre el *log* general de MySQL con objetivos de tipo `network_payload` sobre el tráfico capturado en el puerto 3306, de modo que un mismo *payload* contribuye a varios objetivos por canales independientes y la condición de completitud (`all`) exige cumplir el conjunto.

Tabla 5.1: Cobertura de categorías y monitores por escenario.

Escenario	Categoría	Monitores	Tipos de objetivo
Escalada (sudo vim)	pwning	FS, Proc	• file_created • file_modified • process_running
Inyección SQL	web	Log, Red	• log_pattern • network_payload
Inyección de comandos	web	Red, FS	• network_connection • file_created

Entre los tres escenarios se ejercitaron los cuatro monitores y los principales tipos de objetivo del motor. Con ello se satisfizo el criterio declarado de cubrir al menos tres categorías con detección automática basada en estado, sin que el estudiante deba enviar una *flag* ni el educador deba corregir manualmente.

5.3. Escenarios validados

5.3.1. Escalada de privilegios vía sudo vim (pwning)

El escenario se monta sobre la imagen `flexipwn/vuln-sudo`, que ejecuta un servidor SSH y concede al usuario `ctfuser` permiso `NOPASSWD` para ejecutar `vim` como *root*. El estudiante

se conecta primero al contenedor atacante con las credenciales del panel SSH, salta desde allí al contenedor vulnerable, descubre el privilegio con `sudo -l` y obtiene una *shell* de *root* mediante el *shell-out* de vim (`sudo vim -c '!:bash'`). Con esa *shell* crea un archivo de texto bajo `/root` y modifica `/etc/passwd`, lo que produce evidencia observable de la escalada.

Este escenario dispara tres objetivos: la creación de un archivo bajo `/root` (`file_created`), la modificación de `/etc/passwd` (`file_modified`) y la presencia de un proceso con *euid* 0 cuyo mando es `bash` y cuyo ancestro incluye `sudo` (`process_running`). El monitor de procesos detecta este último en el momento en que vim genera la *shell*. El *walkthrough* completo del estudiante puede encontrarse en el README del repositorio público.

5.3.2. Inyección SQL con *bypass* de *login* (web)

El escenario emplea la imagen `flexipwn/vuln-sqli-mysql`, una aplicación Flask respaldada por MySQL que escucha en el puerto interno 5001 y mantiene activo el *log* general de la base de datos. El estudiante envía dos *payloads* desde el atacante mediante `curl`: un *bypass* clásico de autenticación (`admin' OR '1'='1' --`) y una consulta UNION que extrae datos de una tabla sensible.

La detección combina dos canales. Por un lado, el monitor de *logs* observa el *log* general de MySQL y reconoce patrones como la condición `OR 1=1`, la autenticación exitosa y la consulta sobre la tabla de datos sensibles. Por otro, el *sidecar* de captura ejecuta `tcpdump` con el filtro `port 3306` y reconoce los mismos patrones, además de la *flag* esperada del flujo, directamente en el tráfico hacia MySQL. Esta redundancia ilustra cómo *logs* y red se refuerzan: cada *payload* activa varios objetivos por vías independientes. El *walkthrough* con los dos *payloads* y los seis objetivos asociados está documentado en el README del repositorio.

5.3.3. Inyección de comandos hacia *reverse shell* (web)

El tercer escenario usa la imagen `flexipwn/vuln-command-injection`, una aplicación Flask con un *endpoint* `/ping?host=` vulnerable a inyección de comandos. El estudiante necesita dos sesiones SSH al atacante: en la primera escucha con `nc -lvp 4444`, y en la segunda envía un `curl` que inyecta una *reverse shell* contra ese puerto. Cuando la conexión inversa se establece, el estudiante crea un archivo bajo `/root` desde la *shell* obtenida.

Los objetivos disparados son la conexión de red saliente hacia el puerto 4444 (`network_connection`) y la creación del archivo de evidencia (`file_created`). Este escenario justifica de forma concreta dos decisiones de arquitectura descritas en capítulos anteriores: el *sidecar* de captura `tcpdump` de la Capa 2, necesario para observar la conexión inversa, y la red externa entre atacante y *host* de la Capa 1. Además, como los escenarios anteriores, el *walkthrough* completo está presente en el README del repositorio.

5.4. Pruebas de robustez del sistema

La robustez de FlexiPwn se respalda principalmente con un extenso abanico de pruebas automatizadas. Este contiene 244 funciones de prueba distribuidas en 23 archivos bajo `src/flexipwn/tests/`, que cubren las cuatro capas de la arquitectura: provisión de entornos, monitores, motor de evaluación, la capa de persistencia y *daemon*. Las pruebas de integración que requieren Docker *rootless* en ejecución están marcadas con `pytest.mark.integration` y agrupadas en un archivo aparte, de modo que las pruebas unitarias se ejecutan sin la necesidad de levantar contenedores.

5.4.1. Casos borde cubiertos por pruebas unitarias

Varios de los escenarios de fallo previstos se cubren a nivel unitario, con *mocks* que reproducen la condición sin levantar la infraestructura real. La relación entre cada caso borde y la prueba que lo verifica es la siguiente:

1. **Contenedor caído a mitad del *run*.** Cuando el contenedor vulnerable se detiene durante un *run* activo, el sondeo de los monitores de proceso y de sistema de archivos detecta su ausencia y propaga la transición de estado. Lo verifican `test_layer2_process` y `test_layer2_filesystem`.
2. **YAML malformado.** Una descripción inválida, por ejemplo, sin sección de entorno, con un objetivo sin tipo o con una condición no permitida, es rechazada por la validación de esquema de Pydantic sin dejar una fila parcial en la base de datos. Lo cubre `test_layer3_engine`.
3. **Participante duplicado.** El campo de nombre de usuario es único, de modo que una segunda inserción con el mismo nombre levanta un error de integridad. Lo verifica `test_db.py::TestParticipantCRUD::test_username_unique`.
4. **Colisión de identificador de entorno.** El identificador de entorno es único, dos *runs* con el mismo valor producen un error de integridad en la segunda inserción. Lo cubre `test_db.py::TestEnvIdUnique`.
5. ***Daemon* detenido a mitad del *run*.** Al reiniciarse, el *daemon* reconcilia los *runs* en curso y marca como fallido aquel cuyo contenedor ya no existe. Lo verifica `test_daemon_loop.py::test_reconcile_marks_failed_when_container_missing`.

Cabe recalcar el alcance de estas pruebas. La validación del esquema, por ejemplo, cubre el rechazo del YAML inválido, pero no el mensaje de error que la interfaz de línea de comandos presenta al educador. De la misma forma, el arranque por lotes de múltiples estudiantes está validado a nivel unitario con *mocks*, pero no se ha sometido a carga concurrente real, como se detalla en la sección de trabajo pendiente.

5.4.2. Detección independiente del camino (la “trampa” de tcpdump)

Una propiedad o limitación relevante del monitor de red es que su detección no depende del camino que tome el *payload*. El *sidecar* de captura comparte el espacio de red del contenedor vulnerable y observa todo el tráfico hacia el puerto correspondiente, lo haya generado la aplicación web o no. Para comprobarlo se diseñó un experimento en el que, en lugar de usar el formulario web del escenario de inyección SQL, se envía una consulta SQL directamente al puerto 3306 del contenedor vulnerable desde el atacante.

El resultado del 26 de junio de 2026 registrado en el runbook de pruebas confirmó este comportamiento. MySQL rechazó la conexión del atacante, respondió que el *host* no tenía permiso para conectarse, por lo que la consulta nunca llegó a ejecutarse. Sin embargo, los bytes de la sentencia **SELECT** ya habían viajado por TCP hasta el puerto 3306, y tcpdump los capturó. Los objetivos de red correspondientes pasaron a cumplidos sin que se tocara la aplicación web. El escenario completo no se da por resuelto, porque la condición **all** exige además los objetivos de *log*, que sí requieren el flujo por la aplicación, y la respuesta con la información de una tabla “sensible”, que en este experimento no existió.

En resumen, lo que ocurrió fue lo siguiente: el detector de red reconoce un *patrón* en el tráfico, no la ejecución exitosa del ataque. Se podría argumentar que en ese sentido, el sistema se comporta como una regla de detección tipo SIEM o Sigma, que inspecciona la presencia de indicadores en el tráfico y no el efecto final sobre el objetivo a vulnerar. Pero al mismo tiempo, podría abrir la puerta a falsos positivos si se genera tráfico que coincida con los patrones esperados, aunque no haya logrado comprometer la aplicación. Por ello, es importante que el educador configure adecuadamente los escenarios y los objetivos para minimizar la posibilidad de detecciones erróneas.

5.4.3. Resistencia y contención del entorno

La afirmación de que el modo *rootless* contiene un eventual escape de contenedor, hecha en el Capítulo 4, se respalda con un chequeo de configuración ejecutado sobre la infraestructura del CLCERT el 18 de junio de 2026. El *script* **scripts/containment_check.sh** verifica, de forma estrictamente *read-only*, si los vectores asociados a los escapes recientes de **runc** (CVE-2025-31133, CVE-2025-52565 y CVE-2025-52881) y a la vulnerabilidad Copy Fail (CVE-2026-31431) están cerrados en la configuración desplegada.

El chequeo reportó que esos vectores no son explotables desde el contenedor en la configuración verificada. El modo *rootless* encapsula el *root* del contenedor a un *namespace*, de modo que un escape no aterrizaría como *root* del *host*. La versión de **runc** instalada incorpora los parches correspondientes, y el perfil **seccomp** deniega por defecto la apertura de *sockets* **AF_ALG**, que es la superficie que el *exploit* de Copy Fail necesita para arrancar.

Es importante delimitar el alcance de esta comprobación. Se trata de una verificación de configuración, no de una prueba de concepto que ejecute un *exploit* completo: el *script* no lanza ningún ataque, sino que consulta el estado del runtime y de los controles del *kernel*.

5.5. Pruebas en infraestructura real

5.5.1. Autoprueba de acceso remoto

Para validar el despliegue accesible a distancia, se realizó una autoprueba con el *script* `scripts/test_remote_access.py`. La prueba levantó un entorno publicando el SSH del contenedor atacante únicamente en IPs concretas, como la dirección de la red *overlay* de `netbird` [25] para acceso remoto, en lugar de exponerlo en todas las interfaces. Con el entorno activo, se logró establecer la conexión desde otro dispositivo, simulando el rol de un estudiante que accede al contenedor atacante a distancia.

Hay que notar que se trató de una prueba que no involucró estudiantes reales. El experimento valida que el mecanismo de publicación selectiva por IP funciona y que el acceso remoto a través de la red *overlay* es posible, pero no constituye una evaluación con usuarios del curso.

5.5.2. Trabajo futuro

En primer lugar, no se ha realizado una sesión piloto con un grupo de estudiantes reales ni un despliegue con un curso real. Una prueba de este tipo permitiría medir la experiencia de uso, los tiempos de incorporación y las fricciones del flujo entre educador y estudiante, pero queda fuera del alcance alcanzable en el tiempo disponible.

En segundo lugar, el arranque por lotes de múltiples estudiantes está validado a nivel unitario con *mocks*, no bajo carga concurrente real de un grupo completo de alumnos. Una prueba de carga real mediría los tiempos de espera, los fallos por agotamiento del rango de puertos y la robustez de la base de datos SQLite cuando varios entornos arrancan en paralelo.

En tercer lugar, las pruebas de aislamiento tal que estudiante no pueda resolver los *runs* de otro, y la exposición controlada a la red del DCC quedan como verificaciones pendientes en el servidor del CLCERT.

Por último, más allá de la validación, la arquitectura de capas deja abiertas líneas de crecimiento del sistema. Nuevos escenarios, *targets* y monitores amplían la cobertura de categorías de ejercicio sin modificar el núcleo de evaluación, y nuevos *providers* de aislamiento, como Podman o algún tipo de virtualización, permiten sostener la plataforma sobre otras tecnologías. Entre estas líneas destaca el monitoreo basado en eBPF [26], que habilitaría una observación a nivel de *kernel* más fina y más difícil de eludir que la detección actual. Su exploración excedió el tiempo disponible y se retoma en las conclusiones.

5.6. Discusión

Los tres escenarios validados demuestran que FlexiPwn satisface el criterio de éxito declarado en el Capítulo 1: cubren tres categorías de ejercicio, ejercitan los cuatro monitores

de la Capa 2 y disparan sus objetivos a partir del estado real del sistema, sin *flags* manuales ni corrección del educador. El conjunto de pruebas automatizadas y las verificaciones en la configuración refuerzan esta base, y el experimento de detección independiente del camino permite ver tanto la eficacia del monitor de red como sus límites de interpretación.

En definitiva, lo que queda pendiente concierne sobre todo al uso del sistema en condiciones de aula real: la prueba con estudiantes reales, la carga concurrente de un curso completo y la robustez del sistema verificado en producción.

Capítulo 6

Conclusión

El objetivo de esta memoria fue desarrollar FlexiPwn, una plataforma educativa autohospedable para la práctica de ciberseguridad ofensiva que valida el progreso del estudiante a partir del estado del sistema comprometido, sin recurrir a *flags* manuales. Frente a los objetivos propuestos en la introducción, se cumplieron en su mayoría. Se construyó una arquitectura modular de cuatro capas multi-entorno, con responsabilidades delimitadas y agnóstica al proveedor de entornos. Se ofreció una configuración declarativa en archivos YAML con la que un educador describe objetivos y condiciones de validación. Se implementó un motor de validación automática basado en el estado del sistema. Y se proveyeron herramientas de seguimiento y administración a través de interfaces de línea de comandos, un REPL interactivo y un *daemon* de seguimiento. El objetivo de poner la plataforma a disposición del CLCERT quedó parcial: la herramienta se entregó y se probó su resistencia sobre la infraestructura del laboratorio, pero sin un curso ni usuarios reales que la ejerciten.

A lo largo del trabajo se enfrentaron varios desafíos que dejaron aprendizajes concretos. Crear un sistema de detección de exploits no es sencillo: existen muchas herramientas, pero lo difícil es definir qué observar, porque pueden darse falsos positivos. Además, según cómo se observe una ejecución, una consulta u otra acción, se podría caer en modificar la máquina misma. Esto último se intentó evitar al máximo bajo la idea de agnosticismo entre capas. También generó dificultades la integración de los distintos conocimientos de la carrera, como redes, bases de datos, desarrollo de software y ciberseguridad. Al principio el proyecto podía abrumar, pero los pasos semana a semana disminuían esa sensación. En ese sentido, fue muy grato tener reuniones semanales con el grupo del CLCERT: daban un sentimiento de compañía en este último proyecto universitario e impulsaban a entregar avances al laboratorio cada semana.

La inteligencia artificial se encuentra hoy en un buen punto para el desarrollo de software, y es impresionante lo bien que funciona si se usa correctamente. Un desafío fue calmar la tentación de dejar sin revisar una parte del sistema solo porque “funciona”. Fue importante recordar que la revisión del código resulta crítica para mantener la consistencia de los pilares del trabajo: la verificación automática de exploits, la arquitectura de capas, el máximo agnosticismo entre ellas, la seguridad del sistema, una experiencia de usuario adecuada, la escalabilidad ante más monitores, *targets* y escenarios, y la facilidad de levantar y mantener

la plataforma. Cada vez hay más puestos para ingenieros que incorporan la IA en su flujo de trabajo, y este proyecto apoyó a esa formación, distinguiendo lo que sería el trabajo de un desarrollador “moderno” de los llamados *vibe coders*.

El sistema mantiene limitaciones que conviene reconocer. El monitoreo opera por sondeo (*polling*), de modo que introduce latencia y puede no observar acciones de breve duración. Además, el monitor de red reconoce patrones en el tráfico y no la ejecución exitosa de un ataque, lo que abre la puerta a falsos positivos. Sumado a esto, la plataforma aún no se valida en un aula real con usuarios, por lo que su eficacia pedagógica en uso cotidiano queda por confirmar.

El trabajo futuro pone el énfasis en realizar pruebas realistas del sistema, es decir, su uso en un aula con estudiantes, que es la validación que esta memoria no alcanza a completar. A partir de ahí, la arquitectura habilita la incorporación posterior de más *providers* de aislamiento, escenarios, monitores y *targets*, junto con otras líneas como interfaces alternativas o mecanismos de detección complementarios.

El desarrollo de esta memoria resulta valioso porque expande el abanico de plataformas educativas para enseñar ciberseguridad ofensiva, con gran personalización en un entorno seguro y, sobre todo, con verificación automática de los ejercicios, lo que genera un enfoque más inmersivo para las y los estudiantes que se hagan partícipes. Se espera que la plataforma sea de gran ayuda para los cursos de Taller de Hacking y Ciberseguridad, y que siga creciendo para incluir más y mejores formas de detección de exploits. Si se comenzara de nuevo, se tendría más en consideración el uso de herramientas como eBPF, que podrían haber permitido un alcance más cercano al monitoreo desde el kernel. Se encontró este tipo de herramientas a mitad del desarrollo, por lo que era arriesgado dedicar tiempo a explorarlas en esos momentos.

Bibliografía

- [1] OWASP. OWASP Top 10 2025. https://owasp.org/Top10/2025/0x00_2025-Introduction/, 2025. Visitado el 2025-11-12.
- [2] SANS Institute. Be Ready, Be Resilient: Hands-On Cybersecurity Training & Skill Validation for Real-World Threats. <https://www.sans.org/blog/be-ready-be-resilient-hands-on-cybersecurity-training-skill-validation-for-real-world-threats>, 2025. Visitado el 2025-09-20.
- [3] NepJag. CLCERT: pocFlexiPwn. <https://github.com/NepJag/pocFlexiPwn>, 2025. Visitado el 2025-10-17.
- [4] Sysdig. CVE-2025-31133, CVE-2025-52565, CVE-2025-52881: runc container escape vulnerabilities. <https://www.sysdig.com/blog/runc-container-escape-vulnerabilities>, 2025. Visitado el 2026-05-25.
- [5] skynet-f nvidia. CVE-2025-31133: prueba de concepto de escape de contenedores en runc. <https://github.com/skynet-f-nvidia/CVE-2025-31133>, 2025. Visitado el 2026-06-20.
- [6] TryHackMe. TryHackMe — Cyber Security Training. <https://tryhackme.com/>. Visitado el 2025-08-18.
- [7] Hack The Box. Hack The Box: The #1 Cybersecurity Performance Center. <https://www.hackthebox.com/>. Visitado el 2025-08-18.
- [8] PortSwigger. Web Security Academy: Free misc Training from PortSwigger. <https://portswigger.net/web-security/>. Visitado el 2025-08-18.
- [9] DVWA. Damn Vulnerable Web Application (DVWA). <https://github.com/digininja/DVWA>. Visitado el 2025-08-18.
- [10] CTFd LLC. CTFd: The Easiest Capture The Flag Framework. <https://ctfd.io/>. Visitado el 2025-08-18.
- [11] RootTheBox. RootTheBox. A Game of Hackers (CTF Scoreboard & Game Manager). <https://github.com/moloch--/RootTheBox>. Visitado el 2025-08-25.
- [12] Carnegie Mellon University. picoCTF - CMU Cybersecurity Competition. <https://www.picoctf.org/>. Visitado el 2025-08-25.

- [13] CyberRangeCZ. CyberRangeCZ. <https://github.com/cyberrangecz>. Visitado el 2025-08-25.
- [14] EDURange. EDURange. <https://edurange.org/>. Visitado el 2025-08-25.
- [15] Tectonic Cyber Range. Tectonic Cyber Range. <https://www.fing.edu.uy/inco/proyectos/tectonic>. Visitado el 2025-08-25.
- [16] SigmaHQ. Sigma: Rules – Detection Format. <https://sigmahq.io/docs/basics/rules.html>, 2026. Visitado el 2026-06-30.
- [17] Hack The Box. Best Programming Languages for Cybersecurity. <https://www.hackthebox.com/blog/best-programming-languages-for-cybersecurity>, 2023. Visitado el 2026-05-24.
- [18] Astral. uv: A modern Python package and virtual environment manager. <https://docs.astral.sh/uv/>. Visitado el 2026-05-24.
- [19] Docker. Rootless mode. <https://docs.docker.com/engine/security/rootless/>. Visitado el 2026-05-25.
- [20] Containers. podman-py: Python bindings for Podman. <https://github.com/containers/podman-py>. Visitado el 2026-05-25.
- [21] DEV Community. Docker vs Podman: An In-Depth Comparison (2026). https://dev.to/mechcloud_academy/docker-vs-podman-an-in-depth-comparison-2025-2eia, 2025. Visitado el 2026-05-25.
- [22] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio. An updated performance comparison of virtual machines and Linux containers. In *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2015. pp. 171–172.
- [23] Northflank. Kata Containers vs gVisor. <https://northflank.com/blog/kata-containers-vs-gvisor>, 2026. Visitado el 2026-05-25.
- [24] NepJag. CLCERT: FlexiPwn. <https://github.com/NepJag/FlexiPwn>, 2026. Visitado el 2026-03-04.
- [25] NetBird. NetBird: Open Zero Trust Networking. <https://netbird.io/>. Visitado el 2026-06-20.
- [26] eBPF. eBPF: Dynamically program the kernel for efficient networking, observability, tracing, and security. <https://ebpf.io/>. Visitado el 2026-07-01.